A Project Report

on

CROP RECOMMENDATION SYSTEM

carried out as part of the ***Project Based Learning-3 (DSE3170)***

Submitted

by

Student Name1(Student1 Registration Number)

Student Name2 (Student2 Registration Number)

Student Name3 (Student3 Registration Number)

in partial fulfilment for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering (Data Science)



MANIPAL UNIVERSITY
JAIPUR

School of Computer Science and Engineering
Department of Data Science and Engineering

MANIPAL UNIVERSITY JAIPUR, JAIPUR
RAJASTHAN, INDIA

Nov 2026

CERTIFICATE

This is to certify that the project-based learning , project titled **CROP RECOMMENDATION SYSTEM** is a record of the bonafide work done by **STUDENT NAME 1** (Reg No), **STUDENT NAME 2** (Reg No), **STUDENT NAME 3** (Reg No), submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (Data Science) of Manipal University Jaipur, Jaipur during the academic year 2025-26.

Guide Name

Designation

Project Guide, Department of Data Science and Engineering

Manipal University Jaipur, Jaipur

Class Coordinator Name

Designation

Class instructor, Department of Data Science and Engineering

Manipal University Jaipur, Jaipur

ABSTRACT

Choosing a crop that suits the soil and the local weather is one of the most important decisions a farmer makes each season. Soil test reports and weather data are now easy to obtain, but turning them into a decision is still hard, and a wrong choice can cost a full season of work. Our literature survey showed that most machine learning crop recommenders report the accuracy of a few classifiers and stop there: the models are rarely compared on equal terms, individual recommendations are not explained to the farmer, and fertilizer advice is handled as a separate problem. The objective of this project was to build a crop recommendation system that recommends suitable crops, explains why they were recommended, and suggests how to correct the soil with fertilizer.

We trained and tested ten machine learning algorithms on a public dataset of 2,200 samples covering 22 crops, using seven inputs: soil nitrogen, phosphorus, potassium and pH, and temperature, humidity and rainfall. Random Forest and Naive Bayes gave the highest test accuracy, 99.55% (438 of 440 test samples). We selected Random Forest because it also gives ranked crop probabilities and can be explained exactly, and a gap of only 0.45 percentage points between its training and test accuracy showed that it does not overfit. SHAP and LIME explanations showed that humidity and rainfall influence the recommendation most, followed by N, P and K. For fertilizer recommendation, XGBoost reached 97.69% accuracy on a 99-sample fertilizer dataset, and a nutrient-gap rule compares the farmer's soil with the needs of the recommended crop. The work has been written up as a research paper in IEEE conference format.

LIST OF TABLES

Table No	Table Title	Page No
1.1	Types of intellectual property and their relevance to this project	3
2.1	IEEE conference formatting rules followed in the paper	4
2.2	Sections of the research paper	5
2.3	Datasets used in the project	6
2.4	Summary statistics of the crop dataset	6
2.5	Algorithms and their settings	8
2.6	Software and hardware environment	9
2.7	Crop recommendation results on the 440 test rows and 5-fold cross-validation (CV) on the training set	9
2.8	Top-3 crop recommendations for six farmers from the test set (probability in brackets)	11
2.9	Training and test accuracy of each algorithm (gap in percentage points)	12
2.10	Fertilizers in the fertilizer dataset	15
2.11	Fertilizer recommendation results (stratified 5-fold cross-validation repeated 10 times)	15
2.12	Nutrient-gap advice for the six farmers (gap = crop requirement − soil value)	16
2.13	Criteria used to select a publication venue	17
3.1	Material prepared for the project showcase	20
3.2	Feedback form used at the showcase	20
4.1	Expected questions and where they are answered	22
4.2	Tools used for visual aids	22
4.3	Criteria used to evaluate practice sessions	23

LIST OF FIGURES

Figure No	Figure Title	Page No
1.1	Architecture of the proposed crop recommendation system	2
2.1	First page of the research paper manuscript in IEEE conference format	5
2.2	Data analysis of the crop dataset: (a) correlation between the seven features; (b) mean N, P and K for each crop	7
2.3	Comparison of the ten algorithms: (a) test accuracy (bars) and 5-fold cross-validation accuracy on the training set (dots, mean ± standard deviation); (b) training time on a log scale	9
2.4	Confusion matrix of the Random Forest on the 440 test rows	11
2.5	Overfitting check: (a) training and test accuracy of a decision tree as its depth increases; (b) training and test accuracy of each algorithm	12
2.6	SHAP explanation of the Random Forest: (a) global importance with the share of the total; (b) importance of each feature for each crop, normalised per row, with a dot on the most important feature	13
2.7	Explanation of the recommendation for farmer F1 (mango): (a) SHAP contribution of each input; (b) LIME weights for the same input	14
2.8	Fertilizer recommendation: (a) accuracy of the eight algorithms (mean ± standard deviation over 50 folds); (b) permutation importance of each input for the Random Forest; (c) confusion matrix of XGBoost over one 5-fold run	16
2.9	Steps in preparing, submitting and publishing the research paper	18
4.1	Structure of the 26-slide presentation deck	23

TABLE OF CONTENTS

1. Introduction....1
1.1 Objectives of the Project....2
1.2 Overview of Intellectual Property Rights (IPR) and Patenting....3
2. Publication Preparation and Submission....4
2.1 Research Paper Structure and Formatting Guidelines....4
2.1.1 Structure of the Paper....5
2.1.2 Datasets....6
2.1.3 Data Analysis and Pre-processing....6
2.1.4 Algorithms Used....7
2.1.5 Training, Testing and Evaluation Metrics....8
2.1.6 Results of the Algorithm Comparison....9
2.1.7 Testing the Selected Model....10
2.1.8 Overfitting Analysis....12
2.1.9 Explainable AI Module....13
2.1.10 Fertilizer Recommendation Module....14
2.1.11 Discussion and Limitations....17
2.2 Selection of Journals/Conferences....17
2.3 Submission Process and Peer Review....18
3. Project Showcase....20
3.1 Event Planning and Organization....20
3.2 Audience Engagement and Feedback Collection....20
3.3 Industry Interaction Opportunities....21
4. Presentation Skills Development....22
4.1 Effective Communication Techniques....22
4.2 Visual Aids and Presentation Tools....22
4.3 Practice Sessions and Performance Evaluation....23
5. Outcomes and Achievements....24
5.1 Publications Accepted....24
5.2 Awards and Recognitions....24
6. Conclusion and Future Scope....25
7. References....26
8. Appendices....28
A. Research Paper Manuscript....28
B. Event Photographs and Reports....36

1. Introduction

Agriculture is still the main source of income for a large part of rural India, and for most farming families the year depends on one decision taken before sowing: which crop to grow. Traditionally that choice comes from experience, from what the neighbouring fields grow, or from what fetched a good price last season. Experience is valuable, but it does not always keep up with change. Soil loses or gains nutrients after years of cropping and fertilizer use, and rainfall shifts from one season to the next. A crop that suited a field ten years ago may not suit it today.

At the same time, the information needed for a better decision has become easier to get. Under the Soil Health Card scheme of the Government of India, farmers can have their soil tested for nitrogen (N), phosphorus (P), potassium (K), pH and other properties, and weather data such as temperature, humidity and rainfall are available from public sources. What is missing is a simple way to turn these numbers into advice. A farmer holding a soil test report still has to work out what it means for the coming season.

Machine learning (ML) suits this problem well. Given enough examples of the soil and weather conditions under which different crops are grown, a classifier can learn the pattern and suggest a suitable crop for new conditions [1]. Many student and research projects have built crop recommenders of this kind, usually on a public dataset of soil and weather values, and have reported high accuracy. In this project we built such a system, but we also tried to answer the two questions a farmer would ask next: why was this crop recommended, and what fertilizer does the soil need?

Problem statement. Given the N, P, K and pH values of a soil sample together with the temperature, humidity and rainfall of the area, recommend the crops that best suit these conditions, explain which of the inputs led to the recommendation, and suggest how the soil's nutrients should be corrected with fertilizer.

Literature survey. Liakos et al. [1] reviewed machine learning in agriculture and grouped the published work into crop management, livestock management, water management and soil management; crop recommendation falls in the first group. Van Klompenburg et al. [2] carried out a systematic review of crop yield prediction studies, in which temperature, rainfall and soil type were among the features used most often. This supports our choice of inputs. For choosing a crop, Kumar et al. [3] proposed a crop selection method that picks a sequence of crops to improve the yield rate over a season. Pudumalar et al. [4] combined several classifiers into an ensemble that recommends a crop from soil properties. Doshi et al. [5] built AgroConsultant, a system that compares several classifiers for crop recommendation using soil and climate inputs. The open-source Harvestify project [6] put crop recommendation, fertilizer suggestion and plant disease detection together on one website, and the crop dataset we use was released alongside it [7].

A second line of work deals with explaining ML models. SHAP (SHapley Additive exPlanations) [8] gives each input feature a share of a prediction using Shapley values from game theory, and LIME [9] explains a single prediction by fitting a simple linear model around it. Ryo [10] discussed how such explainable AI methods can be used in agricultural data analysis, where users need to understand a model's output before they act on it.

Research gap. The crop recommendation studies we reviewed usually report the accuracy of a few classifiers on a single split. Models are rarely compared on the same train/test split with more than one metric, individual recommendations are not explained to the farmer, and fertilizer advice is treated as a separate project. A farmer therefore receives a crop name with no reason attached and no guidance on the soil. Our project addresses these three points in one system: a fair comparison of ten algorithms, an

explanation of each recommendation with SHAP and LIME, and a fertilizer module that both predicts a fertilizer and compares the soil with the crop's needs.

Proposed system. Fig. 1.1 shows the system we built. The farmer enters seven values: N, P, K and pH from the soil test, and the temperature, humidity and rainfall of the area. A Random Forest model returns the three most suitable crops with a probability for each. The explanation module shows which of the farmer's values pushed the model towards the first crop. Finally, the fertilizer module suggests a fertilizer and tells the farmer which nutrient to add or skip for that crop.

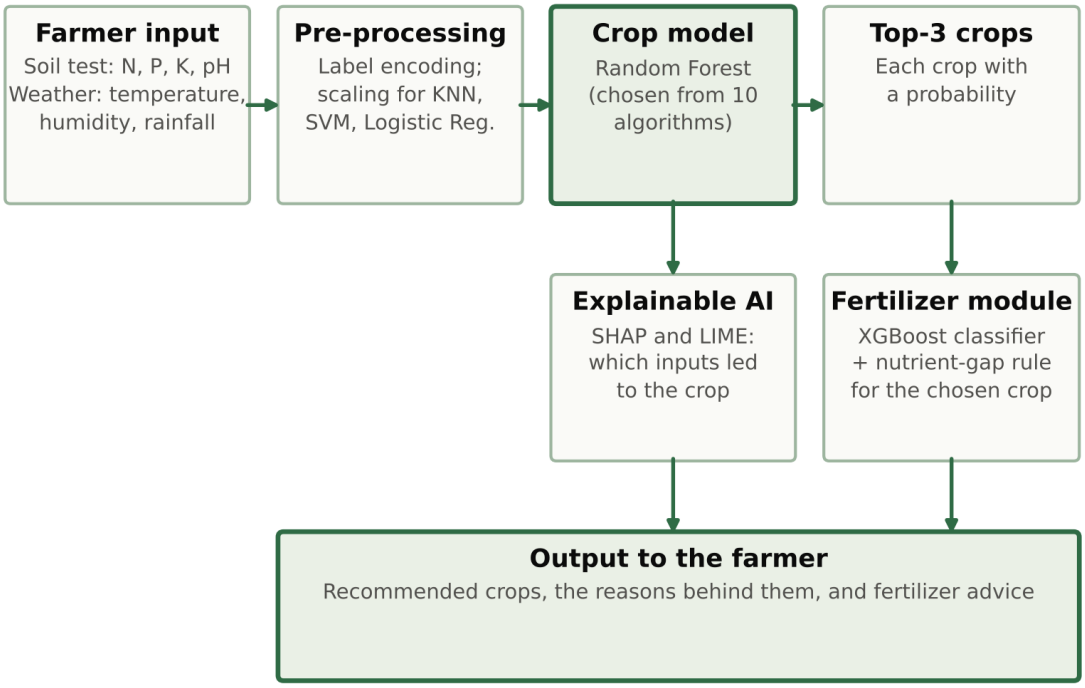


Figure 1.1: Architecture of the proposed crop recommendation system

Organisation of the report. Section 1.1 lists the objectives of the project, and Section 1.2 gives an overview of intellectual property rights and explains why we chose publication rather than a patent. Chapter 2 describes the preparation of our research paper: its format and structure, the datasets, algorithms and results it reports, the choice of venue and the submission process. Chapter 3 covers the project showcase, Chapter 4 the development of our presentation, and Chapter 5 the outcomes of the project. Chapter 6 concludes the report. The references and the full paper manuscript follow at the end.

1.1 Objectives of the Project

The project had the following objectives:

1. To build a crop recommendation model that takes soil nutrient values (N, P, K, pH) and weather values (temperature, humidity, rainfall) as input and returns the most suitable crops with a probability for each.
2. To compare ten machine learning algorithms on the same training and test data, using accuracy, precision, recall, F1-score, cross-validation accuracy and training time, and to select the best model with clear reasons.
3. To check whether the high accuracy of the selected model is a sign of overfitting.
4. To explain each recommendation using SHAP and LIME, both for the dataset as a whole and for an individual farmer.

5. To build a fertilizer recommendation module that predicts a suitable fertilizer from soil and crop data and compares the farmer's soil with the nutrient needs of the recommended crop.
6. To write up the work as a research paper in IEEE conference format and prepare it for submission.

1.2 Overview of Intellectual Property Rights (IPR) and Patenting

Intellectual property rights (IPR) are legal rights given to people over the creations of their minds, such as inventions, written and artistic works, brand names and designs. They let the creator control how the work is used for a fixed period, which encourages people to invest time and money in new ideas. Table 1.1 lists the main types of IPR and how each one relates to our project.

Table 1.1: Types of intellectual property and their relevance to this project

Type	What it protects	Relevance to our project
Patent	A new invention that involves an inventive step and can be applied in industry	Not applicable: the work is a software method (see below)
Copyright	Original literary, artistic and software works, including source code	Applies to our code, report and paper; it arises automatically
Trademark	Names, logos and symbols that identify goods or services	Not applicable: the system has no product name or brand
Industrial design	The shape, pattern or look of a product	Not applicable
Trade secret	Confidential business information	Not applicable: code and data are shared
Geographical indication	Goods linked to a place of origin	Not applicable

In India, patents are granted under the Patents Act, 1970 [11]. To be patentable, an invention must be new, must involve an inventive step that would not be obvious to a person skilled in the field, and must be capable of industrial application. The Act also lists things that are not inventions. Section 3(k) excludes “a mathematical or business method or a computer programme per se or algorithms”. Our project applies known machine learning algorithms (Random Forest, XGBoost and others) and known explanation methods (SHAP and LIME) to public data, and it runs entirely as software. It therefore falls under the Section 3(k) exclusion and is unlikely to be patentable as such. It would also struggle on novelty, since crop recommendation with these algorithms has been published before, as the literature survey shows.

Copyright suits this kind of work better. Under the Copyright Act, 1957 [12], computer programmes are treated as literary works, and copyright exists as soon as the work is created, without registration. Our source code, report, figures and paper are covered in this way.

We also had to respect the rights of others. The crop nutrient table comes from the Harvestify repository [6], which is released under the GNU General Public License version 3 [13]. This licence allows anyone to use, study, share and modify the work, provided that modified versions are shared under the same licence and the source is credited. The crop and fertilizer datasets are public datasets on Kaggle [7], [14]. [Check and state the licence shown on each Kaggle dataset page.] We cite all three sources in the paper and in this report, and we used them for research only.

Because the work is not patentable and is meant to be shared, we chose the publication route. The chapters on patent drafting and patent filing in the report format did not apply to our project and have been removed, as the format instructs. Publishing the paper puts the method and the results on record under our names, and sharing the code lets others check and build on it.

2. Publication Preparation and Submission

The main academic output of the project is a research paper titled “Crop Recommendation System Using Machine Learning, Explainable AI and Fertilizer Recommendation”, written in the IEEE conference format. This chapter explains how the paper is structured and formatted (Section 2.1) and describes in detail the work it reports: the datasets, the algorithms, the experiments and the results. Section 2.2 explains how we selected the venue, and Section 2.3 describes the submission and peer review process. The full manuscript is attached as Appendix A.

2.1 Research Paper Structure and Formatting Guidelines

We prepared the paper in LaTeX using the IEEEtran document class in its conference mode [15], which is the standard template for IEEE conferences. Writing in LaTeX kept the layout, the numbering of equations, figures and tables, and the reference list consistent without manual effort. Table 2.1 summarises the formatting rules we followed, and Fig. 2.1 shows the first page of the manuscript.

Table 2.1: IEEE conference formatting rules followed in the paper

Element	Rule followed
Layout	Two columns; wide figures and tables span both columns
Title and authors	Title centred at the top, followed by author blocks with name, department, institution and e-mail
Abstract and keywords	One-paragraph abstract followed by “Index Terms”
Body text	Times font, 10 pt (the IEEEtran default)
Section headings	Roman numerals for sections (I, II, III), capital letters for subsections (A, B, C)
Figures	Numbered Fig. 1, Fig. 2, … with the caption below the figure
Tables	Numbered with Roman numerals (Table I, Table II, …) with the caption above the table
Equations	Numbered in brackets at the right margin
References	IEEE style, numbered in square brackets in the order of citation
Length	Eight pages including references



Crop Recommendation System Using Machine Learning, Explainable AI and Fertilizer Recommendation

Author One
Dept. of Computer Science and Engineering
University Name
City, India
author1@example.com

Author Two
Dept. of Computer Science and Engineering
University Name
City, India
author2@example.com

Guide Name
Dept. of Computer Science and Engineering
University Name
City, India
guide@example.com

***Abstract*—Choosing a crop that suits the soil and the local weather is one of the first decisions a farmer makes each season, and a poor choice costs a full season of work. We built a crop recommendation system that takes seven inputs (soil nitrogen, phosphorus, potassium and pH, temperature, humidity and rainfall), recommends the three most suitable crops, explains the recommendation, and suggests how to correct the soil with fertilizer. We trained and tested ten machine learning algorithms on a public dataset of 2,200 samples covering 22 crops, using an 80/20 stratified split. Random Forest and Naive Bayes gave the highest test accuracy, 99.55% (438 of 440 test samples), followed by AdaBoost with depth-3 trees (99.32%) and XGBoost (99.09%). AdaBoost with its default settings reached only 17.50%, and we explain why. To show the farmer why a crop was chosen, we used SHAP and LIME on the Random Forest. Humidity (22%) and rainfall (18%) had the largest overall influence, and the two methods named the same top feature for 66% of 100 test cases. For fertilizer recommendation we compared eight algorithms on a 99-sample fertilizer dataset; XGBoost was best with 97.69% accuracy under repeated cross-validation, and the soil N, P and K values carried almost all of the predictive weight. A simple nutrient-gap rule then compares the farmer's soil with the needs of the recommended crop.**

***Index Terms*—Crop recommendation, machine learning, Random Forest, explainable AI, SHAP, LIME, fertilizer recommendation, precision agriculture**

I. Introduction

Most farmers in India decide what to sow from experience and from what their neighbours grow. That works when conditions stay the same, but soil nutrients change with years of cropping and fertilizer use, and rainfall varies from season to season. A crop that does not suit the field gives a low yield, and the loss is felt for the whole season.

Soil testing has become more common, and a farmer can now know the nitrogen (N), phosphorus (P) and potassium (K) content and the pH of the field. Weather data are also easy to obtain. The difficulty is turning these numbers into a decision. Machine learning is a natural fit here: given enough examples of the conditions under which each crop grows well, a model can learn which crop matches a new set of soil and weather values [1].

A recommendation alone is not enough, though. A farmer is unlikely to trust a crop name that comes with no reason, and the next question after "what should I grow?" is usually "what fertilizer should I apply?". We therefore designed the system around three parts: a crop recommender, an explanation of each recommendation, and a fertilizer module.

Our contributions are:

- A comparison of ten algorithms (Logistic Regression, Naive Bayes, K-Nearest Neighbours, SVM, Decision Tree, Random Forest, XGBoost, LightGBM and two AdaBoost variants) on the same train/test split, reporting accuracy, precision, recall, F1-score, cross-validation accuracy and training time.
- An explainable AI module that uses SHAP and LIME to show which soil and weather values led to a recommendation, both for the whole dataset and for a single farmer.
- A fertilizer module with two parts: a comparison of eight algorithms that predict the fertilizer from soil and crop data, and a nutrient-gap rule that compares the farmer's N, P and K with the requirement of the recommended crop.
- All results come from public datasets, which we list with their links so that others can repeat the work.

The rest of the paper is organised as follows. Section II covers the algorithms and related work. Section III describes the datasets and the proposed system. Section IV presents the results, and Section V concludes.

Figure 2.1: First page of the research paper manuscript in IEEE conference format

2.1.1 Structure of the Paper

The paper follows the usual structure of an IEEE conference paper. Table 2.2 lists its sections and what each one contains.

Table 2.2: Sections of the research paper

Section	Content
Abstract	Problem, method and main results in one paragraph
I. Introduction	Motivation, the three parts of the system, contributions
II. Background	The algorithms, SHAP and LIME; related work
III. Proposed System	System overview, datasets, data analysis and pre-processing, algorithms and settings, testing and metrics, explainable AI module, fertilizer module
IV. Results and Discussion	Algorithm comparison, testing of the selected model, explanations, fertilizer results, discussion and limitations
V. Conclusion and Future Work	Summary of findings and next steps
References	21 references in IEEE format

The rest of this section describes the content of the paper in more detail than the eight-page limit allowed, so that the report keeps a full record of the work.

2.1.2 Datasets

We used three public datasets, summarised in Table 2.3.

Table 2.3: Datasets used in the project

Dataset	Source	Rows	Classes	Inputs
Crop Recommendation	Kaggle [7]	2,200	22 crops, 100 each	N, P, K, temperature, humidity, pH, rainfall
Fertilizer Prediction	Kaggle [14]	99	7 fertilizers	Temperature, humidity, moisture, soil type, crop type, N, P, K
Crop nutrient requirements	Harvestify, GitHub [6]	22	One row per crop	Required N, P, K and pH

The crop dataset contains 22 crops: apple, banana, blackgram, chickpea, coconut, coffee, cotton, grapes, jute, kidney beans, lentil, maize, mango, moth beans, mung bean, muskmelon, orange, papaya, pigeon peas, pomegranate, rice and watermelon. Each crop has exactly 100 rows. Every row holds seven values: the N, P and K content of the soil, the soil pH, the temperature in degrees Celsius, the relative humidity in per cent and the rainfall in millimetres.

The fertilizer dataset has 99 rows and seven fertilizers: Urea (22 rows), DAP (18), 28-28 (17), 14-35-14 (14), 20-20 (14), 17-17-17 (7) and 10-26-26 (7). Each row gives the temperature, humidity and soil moisture, the soil type (one of five: loamy, sandy, clayey, black and red), the crop type (one of eleven, such as paddy, wheat, cotton and sugarcane) and the soil's N, P and K.

The third dataset is a small table from the Harvestify repository that lists, for each of the same 22 crops, the N, P and K values the crop needs, together with a suitable pH. It is on the same scale as the crop dataset, which lets us compare a farmer's soil directly with a crop's needs. We checked that the copy we used matches the file in the repository row for row.

We chose these datasets for three reasons. First, the crop dataset uses exactly the inputs a farmer can get today from a soil test and a weather report. Second, it is balanced, so the models are not biased towards common crops. Third, all three datasets are public, so anyone can repeat our experiments with our code.

2.1.3 Data Analysis and Pre-processing

Before training any model we studied the crop dataset. It has no missing values and no duplicate rows. Table 2.4 gives the mean, standard deviation and range of each feature.

Table 2.4: Summary statistics of the crop dataset

Feature	Mean	Std. dev.	Minimum	Maximum
Nitrogen, N	50.55	36.92	0.00	140.00
Phosphorus, P	53.36	32.99	5.00	145.00
Potassium, K	48.15	50.65	5.00	205.00
Temperature (°C)	25.62	5.06	8.83	43.68
Humidity (%)	71.48	22.26	14.26	99.98
pH	6.47	0.77	3.50	9.94
Rainfall (mm)	103.46	54.96	20.21	298.56

The features are on very different scales. Potassium ranges from 5 to 205 and rainfall from about 20 mm to about 299 mm, while pH stays between 3.5 and 9.94. This matters for algorithms that measure distances, such as KNN, and also for SVM and logistic regression, so we standardised the inputs for these three models.

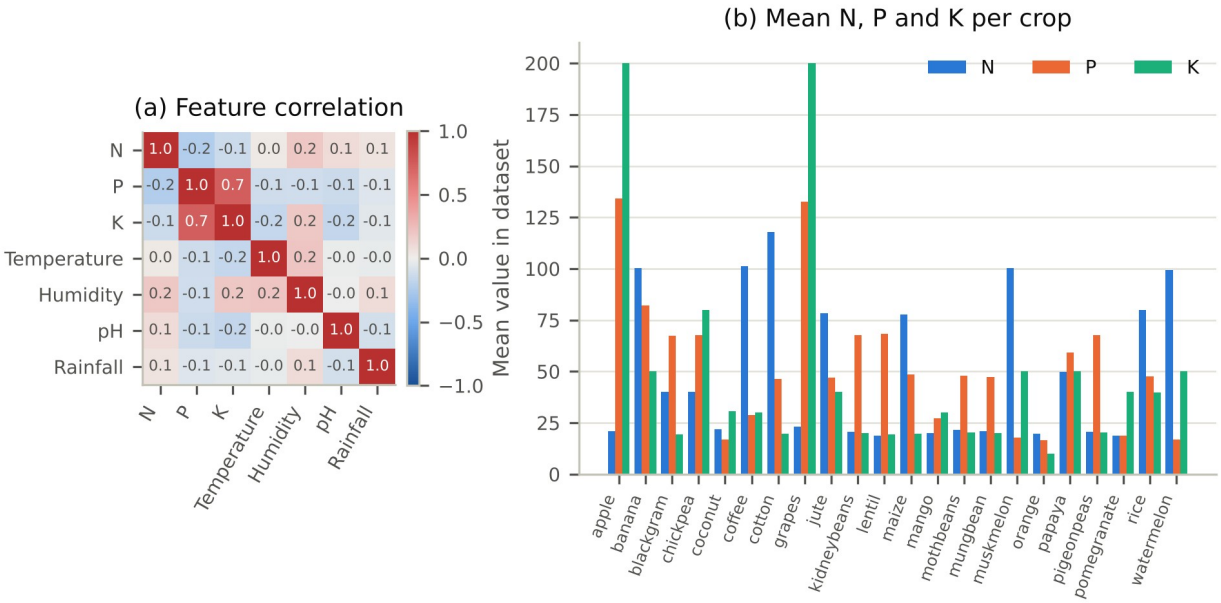


Figure 2.2: Data analysis of the crop dataset: (a) correlation between the seven features; (b) mean N, P and K for each crop

Fig. 2.2(a) shows the Pearson correlation between every pair of features. Only phosphorus and potassium are strongly related (r = 0.74); every other pair lies between −0.23 and 0.21. Each feature therefore carries information of its own, and we kept all seven. Fig. 2.2(b) shows how different the crops are in their nutrient needs. Apple and grapes have by far the highest P and K, at about 133 and 200 on average. Cotton has the highest N (about 118), while crops such as mango and orange are low on all three nutrients. Differences of this size are what the classifiers learn from.

For pre-processing we encoded the 22 crop names as integers and split the data into a training set and a test set with a stratified 80/20 split, using a fixed random seed of 42. This gave 1,760 training rows and 440 test rows, with exactly 20 test rows for each crop. The test rows were kept aside and never used during training. For KNN, SVM and logistic regression we standardised every feature to zero mean and unit variance, fitting the scaler on the training rows only so that no information from the test rows reached the model. The tree-based models and Naive Bayes used the raw values. For the fertilizer dataset, the soil type and crop type columns were one-hot encoded.

2.1.4 Algorithms Used

We compared ten algorithms that cover the main families of classifiers used for crop recommendation. Each is described briefly below, and their settings are listed in Table 2.5.

Logistic Regression fits a linear decision boundary for each crop and converts the result into a probability with the softmax function. It is simple and fast, and it serves as a baseline.

Gaussian Naive Bayes assumes that within each crop every feature follows a normal distribution and that the features are independent of one another. For a new sample x it picks the crop c that maximises

$$P(c) \times \prod_j N(x_j ; \mu_{cj} , \sigma_{cj}^2) \quad (2.1)$$

where μ_{cj} and σ_{cj} are the mean and standard deviation of feature j for crop c.

K-Nearest Neighbours (KNN) [16] keeps all the training samples and labels a new sample by a majority vote of its k closest neighbours. Closeness is measured with the Euclidean distance

$$d(x, x') = \sqrt{(\Sigma_j (x_j - x'_j)^2)} \quad (2.2)$$

We used k = 5. Because the vote depends only on how close samples are, KNN needs scaled features.

Support Vector Machine (SVM) [17] finds the boundary with the widest margin between classes. With a radial basis function (RBF) kernel the boundary can curve, and a problem with many classes is handled by combining several two-class SVMs.

Decision Tree [18] splits the data with a series of yes/no questions on one feature at a time, until each leaf holds mostly one crop. A single tree is easy to read but sensitive to small changes in the data.

Random Forest [19] trains many decision trees, each on a bootstrap sample of the training data and with a random subset of features at each split, and takes a majority vote:

$$\hat{y} = \text{mode}\{ h_1(x), h_2(x), \dots , h_T(x) \} \quad (2.3)$$

where h_t is the t-th tree and T = 100 in our case. The fraction of trees that vote for each crop gives a probability, which we use to rank the crops.

AdaBoost [20] builds a sequence of weak learners and raises the weight of the samples misclassified in one round, so that the next learner concentrates on them. For more than two classes it uses the SAMME algorithm [21]. We tested it in two forms: with its default weak learner, a decision stump with a single split, and with trees of depth 3.

XGBoost [22] and **LightGBM** [23] are gradient boosting libraries in which each new tree is fitted to the errors of the trees built so far. LightGBM grows its trees leaf by leaf and uses histogram-based splits, which makes it fast.

Table 2.5: Algorithms and their settings

Algorithm	Settings
Logistic Regression	Scaled inputs, up to 5,000 iterations
Naive Bayes	Gaussian
KNN	Scaled inputs, k = 5
SVM	Scaled inputs, RBF kernel
Decision Tree	No depth limit
Random Forest	100 trees
XGBoost	300 trees, depth 6, learning rate 0.1
LightGBM	300 trees, 31 leaves, learning rate 0.05
AdaBoost (default)	50 decision stumps (depth 1)
AdaBoost (depth-3 trees)	200 trees of depth 3, learning rate 0.5

All models used the default settings of their libraries apart from the values in Table 2.5. We did not tune hyperparameters, so every algorithm was compared on equal terms; tuning is listed as future work.

2.1.5 Training, Testing and Evaluation Metrics

Each algorithm was trained on the 1,760 training rows and tested once on the 440 held-out rows. We report the following measures. Precision, recall and F1-score are averaged over the 22 crops (macro average).

$$\text{Accuracy} = (\text{number of correct predictions}) / (\text{total number of predictions}) \quad (2.4)$$

$$\text{Precision}_c = TP_c / (TP_c + FP_c) , \quad \text{Recall}_c = TP_c / (TP_c + FN_c) \quad (2.5)$$

$$F1_c = 2 \times \text{Precision}_c \times \text{Recall}_c / (\text{Precision}_c + \text{Recall}_c) \quad (2.6)$$

Here TP_c, FP_c and FN_c are the true positives, false positives and false negatives for crop c. A single split can be lucky or unlucky, so we also ran 5-fold stratified cross-validation on the training set: the training

rows are divided into five parts, each model is trained on four of them and tested on the fifth, and this is repeated five times. We report the mean and standard deviation of the five accuracies. We also measured the training time of each model. Table 2.6 lists the software and hardware we used.

Table 2.6: Software and hardware environment

Item	Version or detail
Programming language	Python 3.13
Machine learning	scikit-learn 1.9.1 [24]
Gradient boosting	XGBoost 3.4.1, LightGBM 4.7.0
Explainable AI	SHAP 0.52.0, LIME 0.2.0.1
Data handling	pandas 3.0.5, NumPy 2.5.3
Plots	Matplotlib 3.11.2
Paper	LaTeX with the IEEEtran class
Hardware	4-core CPU, no GPU
Random seed	42 in every experiment

2.1.6 Results of the Algorithm Comparison

Table 2.7 gives the results of the ten algorithms on the 440 test rows, and Fig. 2.3 shows them as a chart.

Table 2.7: Crop recommendation results on the 440 test rows and 5-fold cross-validation (CV) on the training set

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Errors (of 440)	CV accuracy (%)	Training time (s)
Random Forest	99.55	99.57	99.55	99.55	2	99.38 ± 0.61	0.24
Naive Bayes	99.55	99.59	99.55	99.54	2	99.49 ± 0.42	<0.01
AdaBoost (depth-3 trees)	99.32	99.37	99.32	99.30	3	98.75 ± 0.29	0.92
XGBoost	99.09	99.13	99.09	99.08	4	99.26 ± 0.14	1.02
LightGBM	98.86	98.94	98.86	98.86	5	99.03 ± 0.39	1.44
SVM (RBF)	98.41	98.56	98.41	98.40	7	97.90 ± 1.03	0.16
Decision Tree	97.95	98.06	97.95	97.94	9	98.52 ± 0.68	0.01
KNN (k = 5)	97.95	98.04	97.95	97.93	9	96.53 ± 1.21	<0.01
Logistic Regression	97.27	97.40	97.27	97.25	12	96.82 ± 0.66	0.04
AdaBoost (default)	17.50	11.21	17.50	8.56	363	31.93 ± 5.85	0.14

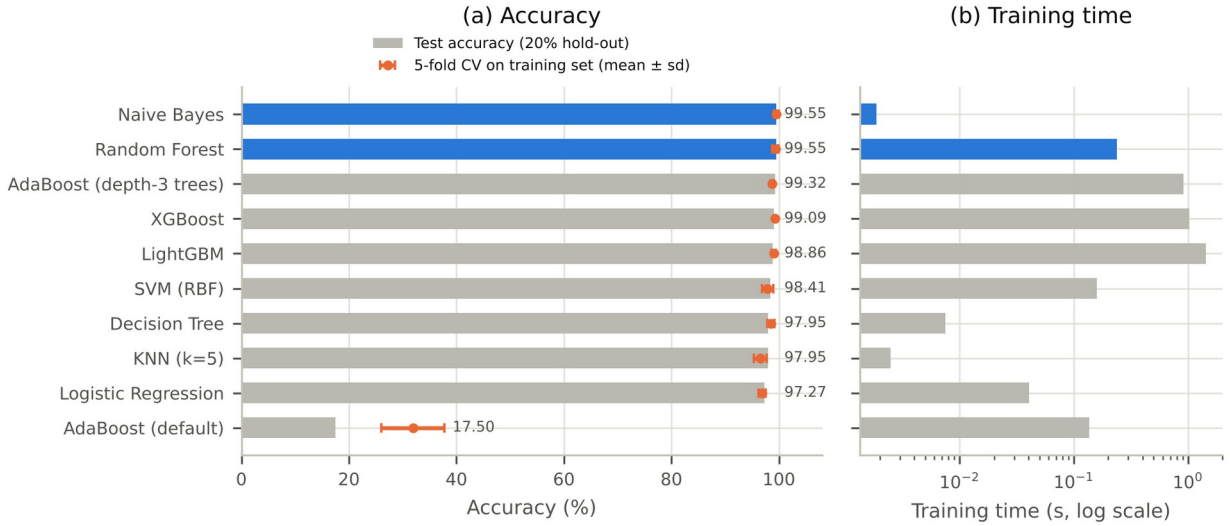


Figure 2.3: Comparison of the ten algorithms: (a) test accuracy (bars) and 5-fold cross-validation accuracy on the training set (dots, mean ± standard deviation); (b) training time on a log scale

Random Forest and Naive Bayes share the top place with a test accuracy of 99.55%; each misclassified only 2 of the 440 test rows. AdaBoost with depth-3 trees (3 errors), XGBoost (4) and LightGBM (5) follow closely. SVM, Decision Tree, KNN and Logistic Regression are a little behind, between 97.27% and 98.41%. The cross-validation results put the same five algorithms at the top, with accuracies between 98.75% and 99.49%, so the ranking is not an accident of one split.

The differences among the top five are very small: between 99.55% and 98.86% there are only three test rows. Accuracy alone cannot separate them, so we chose among them on other qualities. We selected Random Forest for the system for four reasons:

- It ties for the best test accuracy (99.55%) and is stable in cross-validation (99.38 ± 0.61%).
- Its trees vote, so every crop gets a probability, and we can show the farmer a ranked list of three crops instead of a single answer.
- Tree models can be explained exactly and quickly with TreeSHAP [25], which matters for the explanation module.
- It is fast: it trained in 0.24 s and predicts in well under a millisecond per farmer.

Naive Bayes is equally accurate and trains even faster, but explaining it would need KernelSHAP, which is slower and only approximate. We kept Naive Bayes as a backup model.

Why AdaBoost failed with default settings. The one clear failure in Table 2.7 is AdaBoost with its default settings, at only 17.50% test accuracy and 31.93% in cross-validation. Its default weak learner is a decision stump, a tree with a single split. One split can only divide the 22 crops into two groups, so each round of boosting adds very little and the ensemble never learns to separate most crops. When we replaced the stump with a tree of depth 3, which can separate up to eight groups, the same algorithm reached 99.32%. A low AdaBoost score on a problem with many classes therefore points to the choice of weak learner, not to boosting itself.

2.1.7 Testing the Selected Model

Fig. 2.4 shows the confusion matrix of the Random Forest on the test set. Each row is the true crop and each column the predicted crop, so the correct predictions lie on the diagonal. Twenty of the 22 crops were classified perfectly, 20 out of 20 each.

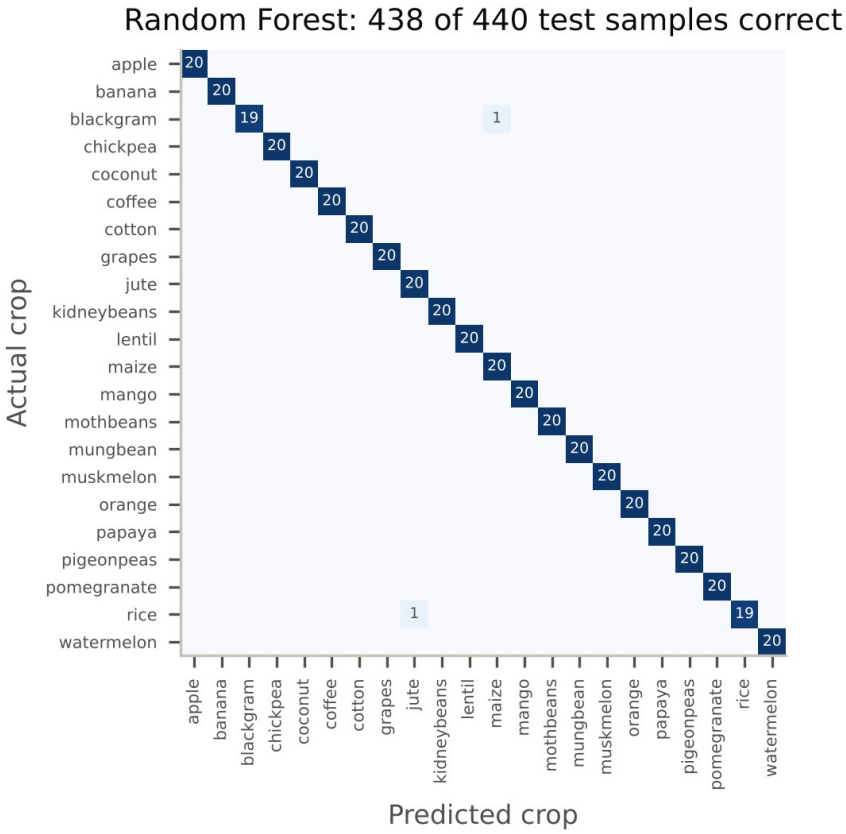


Figure 2.4: Confusion matrix of the Random Forest on the 440 test rows

There were only two errors: one rice sample was predicted as jute, and one blackgram sample as maize. Both mistakes make sense when we look at the data. Rice and jute have almost the same average N, P and K (about 80, 47 and 40), and both grow in humid conditions, with average humidity of 82.3% and 79.6%; they differ mainly in rainfall, 236 mm against 175 mm. Blackgram and maize have the same average humidity (65.1%) and almost the same K (19.2 and 19.8). For these four crops the F1-score is between 0.974 and 0.976; for the other 18 it is 1.00.

To see how the system behaves for individual farmers, we took six rows from the test set, each with a different crop, and treated them as six farmers. Table 2.8 shows their inputs and the three crops the model recommends, with the probability of each.

Table 2.8: Top-3 crop recommendations for six farmers from the test set (probability in brackets)

Farmer	Soil N-P-K	pH	Temp. (°C)	Humidity (%)	Rain (mm)	1st choice	2nd choice	3rd choice
F1	18-17-31	5.7	31.7	45.2	94	mango (0.98)	orange (0.01)	pigeon peas (0.01)
F2	22-44-24	6.0	24.3	56.3	59	moth beans (0.72)	pigeon peas (0.10)	maize (0.08)
F3	3-67-24	5.5	17.0	19.9	103	kidney beans (0.99)	pigeon peas (0.01)	blackgram (0.00)
F4	34-75-24	4.8	23.5	51.3	192	pigeon peas (0.98)	moth beans (0.02)	blackgram (0.00)
F5	76-54-45	6.2	24.3	77.6	185	jute (0.91)	rice (0.08)	papaya (0.01)
F6	117-76-47	6.1	25.6	77.4	93	banana (0.99)	cotton (0.01)	blackgram (0.00)

For all six farmers the first recommendation is the crop actually grown. The probabilities show how sure the model is. For F3 and F6 it is almost certain (0.99). For F2 it is less sure: moth beans has 0.72, followed by pigeon peas (0.10) and maize (0.08). In such a case the ranked list is more useful than a single answer, since the soil and weather suit more than one crop and the farmer can choose based on other things, such as market price or seed availability.

2.1.8 Overfitting Analysis

An accuracy above 99% naturally raises the question of overfitting. A model overfits when it learns the training data too closely, including its noise, and then performs much worse on new data. The usual sign is a large gap between training accuracy and test accuracy. We checked this in three ways.

First, we measured the training and test accuracy of every algorithm on the same split (Table 2.9).

Table 2.9: Training and test accuracy of each algorithm (gap in percentage points)

Algorithm	Training accuracy (%)	Test accuracy (%)	Gap
Logistic Regression	97.39	97.27	0.11
Naive Bayes	99.49	99.55	−0.06
KNN (k = 5)	98.47	97.95	0.51
SVM (RBF)	98.58	98.41	0.17
Decision Tree	100.00	97.95	2.05
Random Forest	100.00	99.55	0.45
XGBoost	100.00	99.09	0.91
LightGBM	100.00	98.86	1.14
AdaBoost (default)	17.44	17.50	−0.06
AdaBoost (depth-3 trees)	99.55	99.32	0.23

The gap is below one percentage point for most algorithms and only 0.45 points for the Random Forest. The largest gap is 2.05 points, for the single decision tree, which is expected because a tree with no depth limit fits its training data exactly. For Naive Bayes the test accuracy is even slightly higher than the training accuracy.

Second, we trained decision trees of increasing depth and recorded the training and test accuracy of each (Fig. 2.5(a)). In a model that overfits, the two curves separate as the model becomes more complex. Here they rise together: at depth 8 the tree scores 89.49% on training and 87.05% on test, and with no depth limit 100% and 97.95%.

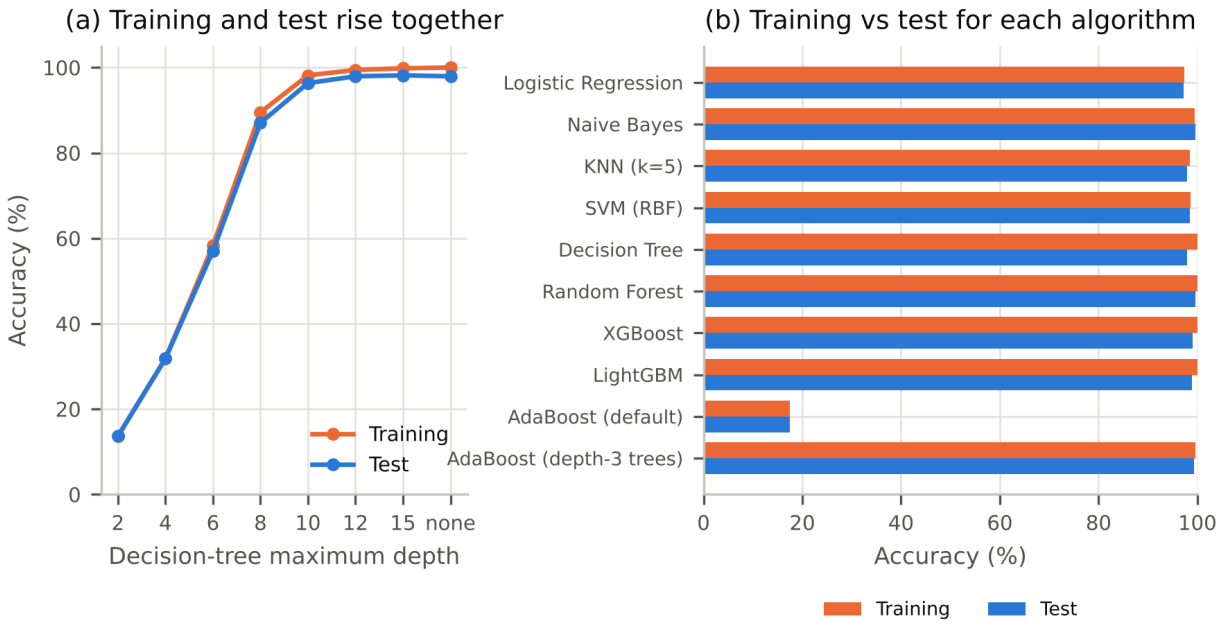


Figure 2.5: Overfitting check: (a) training and test accuracy of a decision tree as its depth increases; (b) training and test accuracy of each algorithm

Third, we ran a label-permutation test [26]. We shuffled the crop labels of the training set at random, retrained the Random Forest and measured its 5-fold cross-validation accuracy, and repeated this 100 times. With shuffled labels the model scored 4.61% on average and never more than 5.85%, which is what random guessing gives for 22 crops (1/22 = 4.55%). With the real labels it scored 99.38% (p = 0.0099, the smallest value that 100 permutations allow). The model therefore learns a real relationship between the inputs and the crop, not noise.

The high accuracy comes from the dataset rather than from the model. The crops occupy clearly separated ranges of N, P, K, humidity and rainfall, so almost any reasonable algorithm can tell them apart; this is why even Naive Bayes, which trains in a few milliseconds, reaches 99.55%. The dataset's author built the N, P and K values from a typical value for each crop with a small random variation [6]. Our results therefore show that the models learn this dataset very well. Testing on real field data is needed before the same accuracy can be expected on farms; we return to this point under limitations.

2.1.9 Explainable AI Module

A recommendation that comes with no reason is hard to trust. The explanation module uses SHAP [8] and LIME [9] to show which inputs led the Random Forest to its recommendation.

SHAP splits one prediction into a contribution from each input:

$$f(x) = \varphi_0 + \Sigma_j \varphi_j \tag{2.7}$$

where φ_0 is the average output of the model and φ_j is the contribution of feature j, computed as a Shapley value from cooperative game theory. For tree models, TreeSHAP computes these values exactly [25]. The global importance of feature j is the mean absolute SHAP value over the n test rows and the K = 22 crop outputs:

$$I_j = (1 / nK) \Sigma_i \Sigma_k | \varphi_j^{(k)}(x_i) | \tag{2.8}$$

LIME works differently. It creates many slightly changed copies of the farmer's input, asks the model to predict each one, and fits a simple weighted linear model to those predictions. The weights of that linear model are the explanation. We used 2,000 perturbed samples for each explanation.

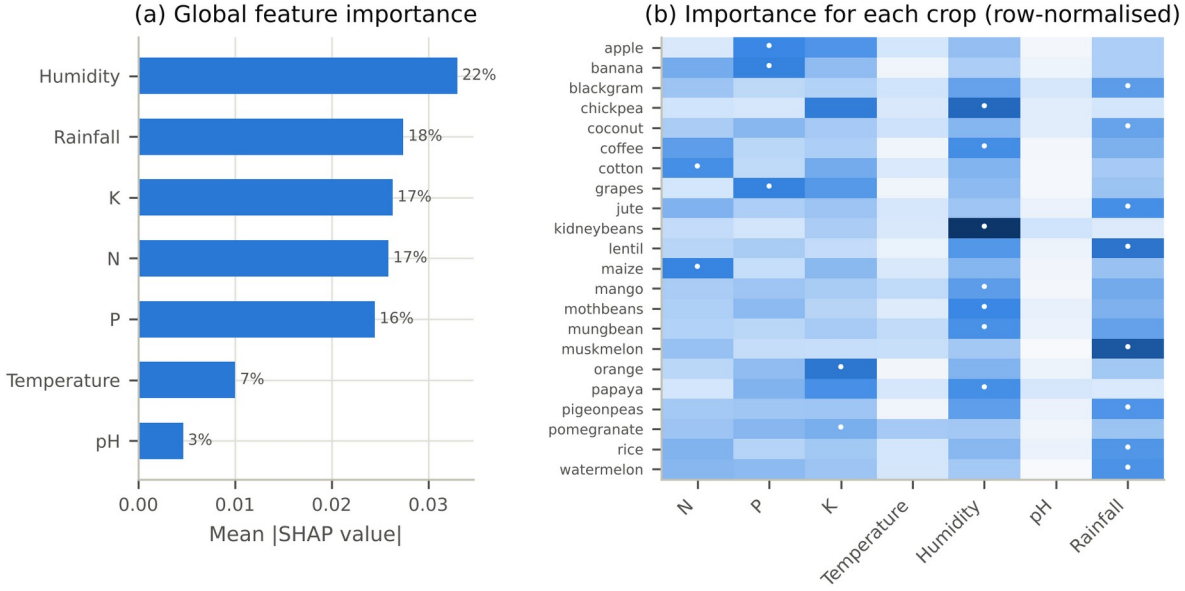


Figure 2.6: SHAP explanation of the Random Forest: (a) global importance with the share of the total; (b) importance of each feature for each crop, normalised per row, with a dot on the most important feature

Fig. 2.6(a) shows the global importance. Humidity has the largest influence, with 22% of the total, followed by rainfall (18%), K and N (17% each) and P (16%). Temperature (7%) and pH (3%) matter much less. The model therefore relies on a mix of weather and soil values, not on a single dominant feature.

Fig. 2.6(b) is more useful for farmers, because it shows that the deciding factor changes from crop to crop. We computed it from each crop's own test rows and its own output. Rainfall is the most important feature for 8 crops, among them rice, jute, lentil and watermelon. Humidity is the most important for 7, among them chickpea, kidney beans and mango. Phosphorus decides apple, banana and grapes; nitrogen decides cotton and maize; and potassium decides orange and pomegranate. These results agree with the data analysis in Fig. 2.2(b), where cotton stands out for its high N, and apple and grapes for their high P and K.

Fig. 2.7 explains a single recommendation, for farmer F1 of Table 2.8, who was advised to grow mango with probability 0.98.

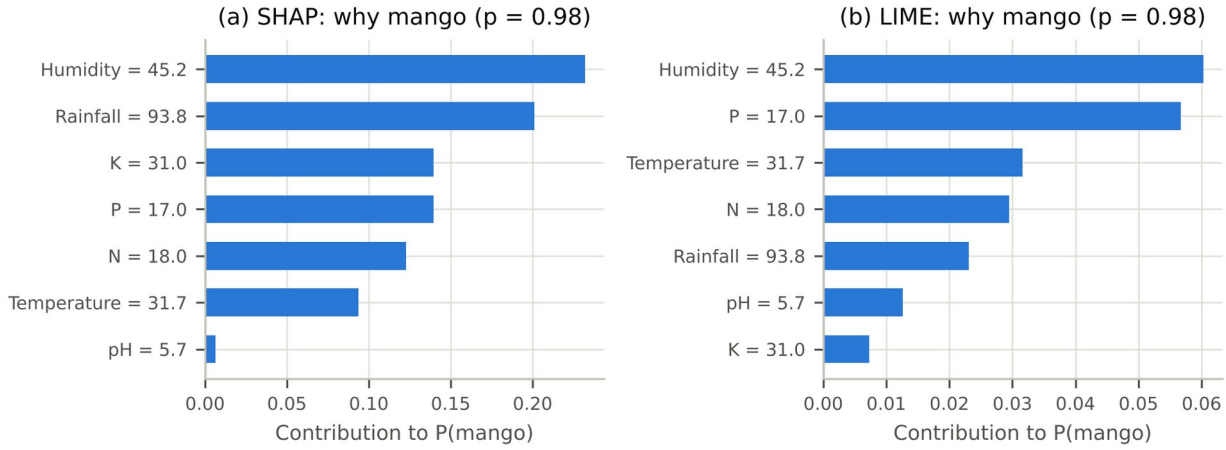


Figure 2.7: Explanation of the recommendation for farmer F1 (mango): (a) SHAP contribution of each input; (b) LIME weights for the same input

SHAP gives the largest contributions to humidity (45.2%, contribution 0.23) and rainfall (93.8 mm, contribution 0.20), followed by K, P and N. Put simply, the field is fairly dry and warm (31.7 °C) and the soil is low in all three nutrients, which is the typical setting for mango in the data. LIME also puts humidity first and expresses it as the rule “humidity ≤ 60.20”, but it ranks phosphorus second and rainfall fifth. The two methods agree on the main reason but not on every detail. To measure this, we compared them on 100 randomly chosen test rows: they named the same most important feature in 66% of cases, and on average they shared 78% of their top three features. For this reason the system shows the farmer only the two or three main reasons, where the methods mostly agree, rather than a full ranking.

2.1.10 Fertilizer Recommendation Module

The fertilizer module has two parts: a machine learning model that predicts a fertilizer from soil and crop data, and a nutrient-gap rule that compares the farmer's soil with the needs of the recommended crop.

Fertilizers in the dataset. The name of most fertilizers in the dataset is its grade: three numbers giving the percentage of nitrogen (N), phosphate (P_2O_5) and potash (K_2O) in the product. Table 2.10 lists the seven fertilizers.

Table 2.10: Fertilizers in the fertilizer dataset

Fertilizer	Grade (N-P2O5-K2O)	Main nutrients supplied	Rows
Urea	46-0-0	Nitrogen only	22
DAP (diammonium phosphate)	18-46-0	Mostly phosphorus, some nitrogen	18
28-28	28-28-0	Equal nitrogen and phosphorus	17
14-35-14	14-35-14	High phosphorus with nitrogen and potassium	14
20-20	20-20-0	Equal nitrogen and phosphorus, lower strength	14
17-17-17	17-17-17	Balanced nitrogen, phosphorus and potassium	7
10-26-26	10-26-26	Phosphorus and potassium with some nitrogen	7

Machine learning model. We trained eight algorithms to predict the fertilizer from the eight inputs. With only 99 rows, a single 80/20 split would leave about 20 test rows, and only one or two for the rarest fertilizers. We therefore used stratified 5-fold cross-validation repeated 10 times, which gives 50 test folds, and report the mean and standard deviation of the accuracy and the mean macro F1-score (Table 2.11).

Table 2.11: Fertilizer recommendation results (stratified 5-fold cross-validation repeated 10 times)

Algorithm	Accuracy (%)	Macro F1-score (%)
XGBoost	97.69 ± 3.20	95.02
Decision Tree	96.28 ± 4.00	92.24
Random Forest	95.07 ± 4.43	89.22
Logistic Regression	94.35 ± 4.56	87.59
SVM (RBF)	88.09 ± 5.19	77.61
AdaBoost (depth-3 trees)	86.19 ± 12.09	74.14
KNN (k = 5)	71.41 ± 10.40	63.13
Naive Bayes	66.74 ± 9.59	60.58

XGBoost gave the highest accuracy, 97.69 ± 3.20%, followed by the Decision Tree (96.28%), Random Forest (95.07%) and Logistic Regression (94.35%). KNN (71.41%) and Naive Bayes (66.74%) performed poorly. The standard deviations are larger than for the crop model, between 3 and 12 points, because each test fold holds only about 20 rows. We selected XGBoost for the fertilizer module because it had the highest accuracy, the highest F1-score and the smallest spread.

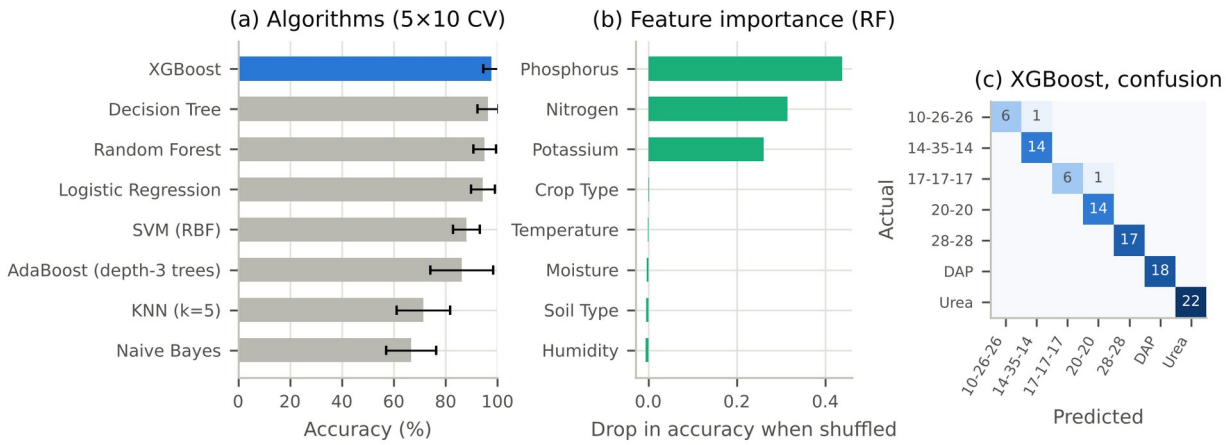


Figure 2.8: Fertilizer recommendation: (a) accuracy of the eight algorithms (mean ± standard deviation over 50 folds); (b) permutation importance of each input for the Random Forest; (c) confusion matrix of XGBoost over one 5-fold run

To find out which inputs the model relies on, we computed permutation importance for the Random Forest on held-out folds, that is, the drop in accuracy when the values of one input are shuffled (Fig. 2.8(b)). Shuffling phosphorus lowers the accuracy by 0.44, nitrogen by 0.31 and potassium by 0.26, while shuffling crop type, temperature, moisture, soil type or humidity changes it by 0.006 or less. In this dataset, then, the choice of fertilizer depends almost entirely on the soil's N, P and K. This makes sense, since a fertilizer is chosen to supply the nutrients the soil lacks. It also explains the weak results of KNN and Naive Bayes: both treat all eight inputs as equally relevant, while only three of them carry the signal. In one 5-fold run XGBoost misclassified 2 of the 99 rows (Fig. 2.8(c)): one 10-26-26 row was predicted as 14-35-14 and one 17-17-17 row as 20-20. Both pairs are complex fertilizers that supply similar nutrients.

Nutrient-gap rule. The ML model names a fertilizer, but it does not tell the farmer why. The second part of the module follows the approach used in the Harvestify project [6]. For a recommended crop c with required nutrients r_c = (N, P, K) and the farmer's soil values s, we compute the gap

$$g = r_c - s\,, \qquad j* = \operatorname{argmax}_j |\, g_j \,| \tag{2.9}$$

The nutrient $j*$ with the largest absolute gap decides the advice. If $g_{j*} > 0$, the soil is short of that nutrient, and we suggest a fertilizer that supplies it: urea for nitrogen, DAP or single super phosphate (SSP) for phosphorus, and muriate of potash (MOP) for potassium. If $g_{j*} < 0$, the soil already holds more of that nutrient than the crop needs, and we advise skipping it for the season, which also saves the farmer money. Table 2.12 applies the rule to the six farmers of Table 2.8, for their first and second recommended crops.

Table 2.12: Nutrient-gap advice for the six farmers (gap = crop requirement − soil value)

Farmer	Soil N-P-K	Rank	Crop	Required N-P-K	Largest gap	Advice
F1	18-17-31	1	mango	20-20-30	P low by 3	Add phosphate (DAP or SSP)
F1	18-17-31	2	orange	20-10-10	K high by 21	Skip potash this season
F2	22-44-24	1	moth beans	20-40-20	P high by 4	Skip phosphate this season
F2	22-44-24	2	pigeon peas	20-60-20	P low by 16	Add phosphate (DAP or SSP)
F3	3-67-24	1	kidney beans	20-60-20	N low by 17	Add nitrogen (urea)
F3	3-67-24	2	pigeon peas	20-60-20	N low by 17	Add nitrogen (urea)
F4	34-75-24	1	pigeon peas	20-60-20	P high by 15	Skip phosphate this season
F4	34-75-24	2	moth beans	20-40-20	P high by 35	Skip phosphate this season
F5	76-54-45	1	jute	80-40-40	P high by 14	Skip phosphate this season
F5	76-54-45	2	rice	80-40-40	P high by 14	Skip phosphate this season

Farmer	Soil N-P-K	Rank	Crop	Required N-P-K	Largest gap	Advice
F6	117-76-47	1	banana	100-75-50	N high by 17	Skip nitrogen this season
F6	117-76-47	2	cotton	120-40-20	P high by 36	Skip phosphate this season

For the first recommended crop the gaps are mostly small. F1's largest gap for mango is only 3 units of P, and F2's is 4 units for moth beans. This is expected, because the crop model picks the crop whose usual conditions are closest to the farmer's field. The second crop usually needs more correction: F1's soil has 21 units more K than orange needs, and F6's soil has 36 units more P than cotton needs. F3 is an exception. Its soil has only 3 units of N against the 20 that kidney beans need, so the system advises a nitrogen fertilizer such as urea. The rule therefore also tells the farmer how much the soil would have to change if they preferred a different crop.

2.1.11 Discussion and Limitations

Three points come out of these results. First, on this dataset the best algorithms are very close: Random Forest, Naive Bayes, AdaBoost with depth-3 trees and XGBoost all exceed 99% test accuracy. Reporting only the single highest number would hide this; the cross-validation results and the error counts make it clear. Second, the explanations make the recommendations easier to check. The per-crop importance agrees with what the data show about each crop, and a farmer can see which of their own values led to the advice. Third, the fertilizer models confirm that the soil's N, P and K drive the choice of fertilizer, and the nutrient-gap rule turns this into advice tied to the crop the farmer is about to sow.

The work also has limitations, which we state openly in the paper:

- The crop dataset was assembled by its author from typical values for each crop and weather records, and the author notes that it should not be used for real farming decisions [6]. Our accuracies show how well the algorithms learn this dataset; they are not yet a measure of performance on farmers' fields.
- The fertilizer dataset is small, with 99 rows, and two of its fertilizers have only seven rows each.
- Neither dataset states the units of N, P and K, so the nutrient-gap advice names a nutrient and a direction, not a quantity in kg/ha.
- All algorithms used default settings, and tuning could change the order among the closely matched top models.

2.2 Selection of Journals/Conferences

Choosing where to submit is part of preparing a paper. A good venue should cover the topic of the paper, follow a fair peer review process, and be indexed in databases where other researchers will find the work. We considered both conferences and journals, using the criteria in Table 2.13.

Table 2.13: Criteria used to select a publication venue

Criterion	What we looked for
Scope	Machine learning applications, smart or precision agriculture, or explainable AI
Indexing	Proceedings published in IEEE Xplore or indexed in Scopus
Peer review	A clear review process with written comments from reviewers
Format	Accepts papers in the IEEE conference format, about 6 to 8 pages
Timeline	Submission deadline and decision date that fit the semester
Cost and travel	Affordable registration fee; presentation in person or online
Credibility	Organised or sponsored by a recognised body; avoid venues that promise acceptance for a fee

Conferences suit a student project well. They review papers faster than journals, and the presentation brings direct feedback from other researchers. IEEE-sponsored international conferences held in India regularly include tracks on machine learning applications and smart agriculture, and they accept papers in exactly the format we used. Journals give a more detailed review and a permanent record, but the review usually takes several months. Journals that publish work of this kind include Computers and Electronics in Agriculture, which published one of the reviews we cite [2], and Artificial Intelligence in Agriculture, which published the explainable AI study we cite [10]. Since our results come from public benchmark data and still need field validation, we decided that a conference was the right first step, with a journal version to follow once the system has been tested on real field data.

Based on these criteria we selected [name of the conference, organiser, dates and mode]. [Add one or two lines on why this conference was chosen, for example its scope and indexing.]

2.3 Submission Process and Peer Review

Submitting a paper follows a fixed sequence of steps, shown in Fig. 2.9.

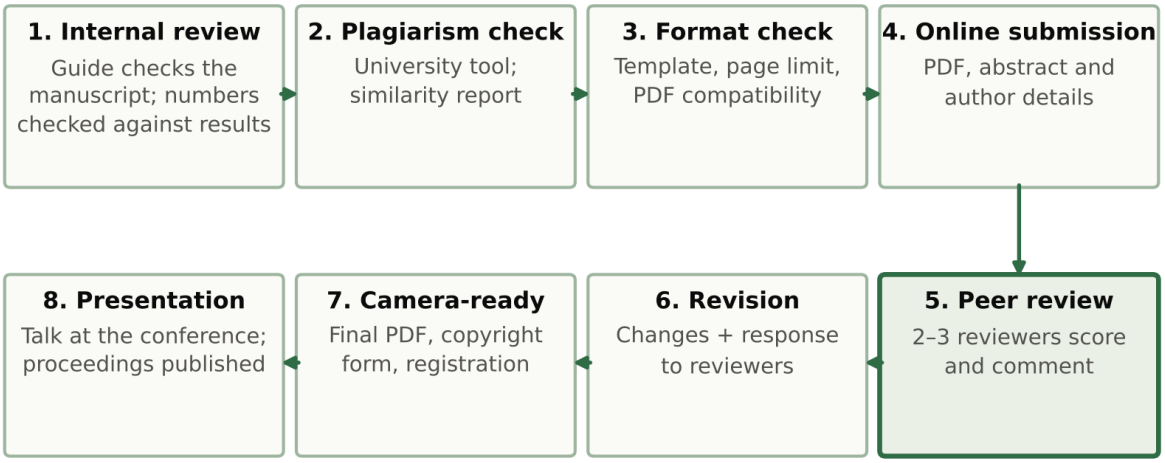


Figure 2.9: Steps in preparing, submitting and publishing the research paper

1. **Internal review.** Before submission, the manuscript was reviewed by our project guide [guide name]. We checked every number in the paper against the result files produced by our code, using a ledger that maps each number to its source file.
2. **Plagiarism check.** The manuscript is checked with the plagiarism tool provided by the university, as the PBL guidelines require. [Add the similarity percentage reported.]
3. **Formatting check.** We confirmed that the paper follows the conference template and the page limit. Many IEEE conferences also require the final PDF to pass the IEEE PDF eXpress check, which confirms that fonts are embedded and the file can be added to IEEE Xplore.
4. **Online submission.** Papers are submitted through the conference's online system, commonly Microsoft CMT or EasyChair, with the title, abstract, keywords, author details and the PDF. [Add the submission date and paper ID.]
5. **Peer review.** The paper is sent to reviewers, usually two or three experts, who score it on originality, technical quality, clarity and relevance and write comments. Many conferences use double-blind review, in which neither the authors nor the reviewers know each other's names, so the author block must be removed from the review copy if the conference asks for this.

6. **Revision.** If the paper is accepted, the authors revise it according to the reviewers' comments and write a short response explaining each change. We have already prepared answers to the comments we expect: the question of overfitting (Section 2.1.8), the choice of Random Forest over the other models (Section 2.1.6) and the use of a public benchmark dataset instead of field data (Section 2.1.11).
7. **Camera-ready submission and copyright.** The final version is uploaded together with the IEEE copyright form, and at least one author registers for the conference.
8. **Presentation and publication.** The paper is presented at the conference, and the proceedings are published, in IEEE Xplore for IEEE conferences.

Current status of our paper: [submitted / under review / accepted, with the date; once the reviews arrive, add a short summary of the reviewers' comments and how they were addressed].

3. Project Showcase

The project showcase is where we present the working system to faculty, fellow students and visitors. This chapter describes how we prepared for it, how we planned to engage the audience and collect feedback, and the opportunities we see for working with industry.

3.1 Event Planning and Organization

[Add the name, date and venue of the showcase event, and the time slot given to the team.] We prepared the material listed in Table 3.1.

Table 3.1: Material prepared for the project showcase

Item	Purpose	Details
Demonstration	Walk through one farmer's case from soil test to fertilizer advice	Our script 12_crop_system.py regenerates every result and graph in about two minutes on a 4-core CPU, without internet access
Presentation deck	Explain the project in a fixed time	26 slides in six sections with speaker notes (Section 4.2)
Research paper	Show the written results to evaluators	Printed copy of the eight-page IEEE manuscript (Appendix A)
Result graphs	Support questions about specific results	Printed charts of the algorithm comparison, confusion matrix, SHAP explanations and fertilizer results

Our plan for the event had three parts. We kept the demonstration offline, so that it would not depend on the internet at the venue. We kept printed copies of the key results in case the projector or laptop failed. And we prepared a short and a long version of the talk, so that we could fit whatever time slot we were given. The work of presenting was divided among the team as follows: [Student 1] explains the problem and the datasets, [Student 2] the algorithms and results, and [Student 3] the explainable AI and fertilizer modules. [Edit to match your actual roles.]

3.2 Audience Engagement and Feedback Collection

A crop recommender is easy for visitors to relate to, so we built the presentation around one farmer. We begin with farmer F1's soil test and weather values, show the recommended crops and their probabilities, then the SHAP explanation, and finally the fertilizer advice (Section 2.1.10). Visitors can then pick another farmer from Table 2.8 and see how the advice changes. When someone asks whether 99.5% accuracy means overfitting, we show the training and test accuracies of Table 2.9 and the result of the shuffled-label test.

To collect feedback we prepared a short form with the questions in Table 3.2.

Table 3.2: Feedback form used at the showcase

No.	Question	Answer type
1	How clearly did the team explain the problem?	Rating 1–5
2	How easy was it to understand why a crop was recommended?	Rating 1–5
3	Would a farmer trust the recommendation more with the explanation shown?	Yes / No / Not sure
4	Was the fertilizer advice clear and practical?	Rating 1–5
5	What would make the system more useful for farmers?	Free text
6	Any other comments or suggestions?	Free text

Summary of the feedback received: [add the number of responses, the average ratings and the main suggestions after the event].

3.3 Industry Interaction Opportunities

The system could be useful to several groups outside the university, and each of them could also help us improve it:

- Krishi Vigyan Kendras (farm science centres) and state agriculture departments, which advise farmers and could provide real soil test results for validating the model.
- Agri-tech companies that offer farm advisory apps, where a crop and fertilizer recommender with explanations could be one feature.
- Fertilizer cooperatives and dealers, for whom the nutrient-gap rule could support advice on which product to buy and which to skip.
- Soil testing laboratories, which produce the N, P, K and pH values the system needs.

The most valuable contribution from any of these partners would be real field data: soil test reports linked to the crops actually grown and their yields. That would address the main limitation of the current work. [Add any interaction that took place, such as a visit, a meeting or an e-mail exchange, with the date and outcome.]

4. Presentation Skills Development

The project had to be presented to very different audiences: faculty who wanted technical detail, fellow students, and, in the future, farmers who need plain advice. This chapter describes how we prepared for this.

4.1 Effective Communication Techniques

We used the following techniques while preparing the talk and the slides:

- **One idea per slide.** Each slide answers one question, such as “Why did we choose Random Forest?” or “Why did AdaBoost fail?”, and its title states that question or its answer.
- **Numbers with context.** Instead of saying only “99.55% accuracy”, we say “438 of 440 test samples correct” and add that the top five models differ by only three test samples. Numbers given this way are easier to remember and harder to misread.
- **Start from the user.** We explain the system through one farmer's journey, from soil test to fertilizer advice, before showing the technical results.
- **Plain language first.** Terms such as SHAP, cross-validation and overfitting are explained in one sentence the first time they appear, for example “SHAP measures how much each input pushed the model towards a crop”.
- **Honest limits.** We state clearly what the results do not show, in particular that the crop dataset is not field data. Raising this ourselves builds more trust than having it raised by the audience.
- **Prepared answers.** We listed the questions we expected and prepared a short answer and supporting material for each (Table 4.1).

Table 4.1: Expected questions and where they are answered

Expected question	Short answer	Supporting material
Is 99.5% accuracy overfitting?	No: training and test accuracy differ by 0.45 points for Random Forest, and shuffled labels give chance-level accuracy	Section 2.1.8, Table 2.9, Fig. 2.5
Why Random Forest and not Naive Bayes?	Equal accuracy, but Random Forest gives a ranked list and exact SHAP explanations	Section 2.1.6
Why did AdaBoost score only 17.50%?	Its default one-split stumps are too weak for 22 crops; with depth-3 trees it reaches 99.32%	Section 2.1.6
Do SHAP and LIME agree?	On the top reason in 66% of cases, so only the top two or three reasons are shown	Section 2.1.9, Fig. 2.7
Will it work on real farms?	Not yet shown; field validation with Soil Health Card data is the next step	Section 2.1.11, Chapter 6
How much fertilizer should be applied?	The current rule names the nutrient and the direction; amounts in kg/ha are future work	Section 2.1.10

4.2 Visual Aids and Presentation Tools

Table 4.2 lists the tools we used to prepare the visual material.

Table 4.2: Tools used for visual aids

Tool	Used for
Matplotlib (Python)	All result graphs: data analysis, algorithm comparison, confusion matrix, SHAP and fertilizer charts, saved at 300 dpi
SHAP and LIME libraries	The explanation values behind Fig. 2.6 and Fig. 2.7
LaTeX with IEEEtran	Layout of the research paper, its equations and tables

Tool	Used for
Online slides tool	The 26-slide presentation with speaker notes, exportable to PowerPoint and PDF
Word processor	This report

The deck has 26 slides in six sections (Fig. 4.1). Every slide uses the same layout: a short label at the top, a title that states the point of the slide, and the figure or table below it. We used one colour family throughout, with a dark green for headings and highlights on a light background, so that the charts stand out. Each slide carries speaker notes in plain language, which we used for rehearsal.

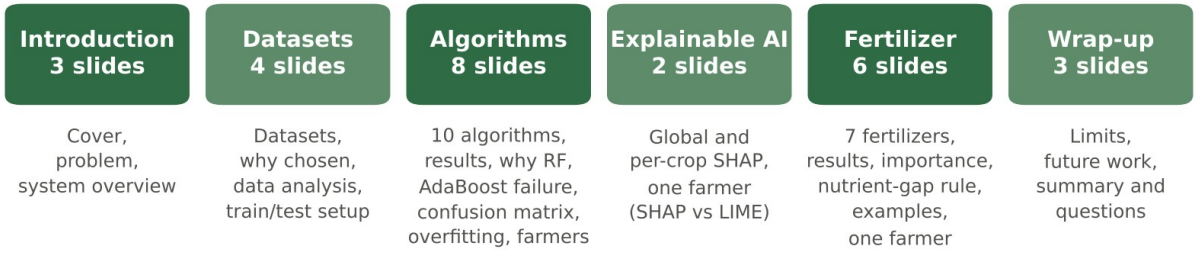


Figure 4.1: Structure of the 26-slide presentation deck

For the charts we followed a few rules. The best models are highlighted in one colour and the rest are shown in grey; axes are labelled with their units; and each chart has a one-line takeaway on the slide, so that the audience does not have to work out the message alone.

4.3 Practice Sessions and Performance Evaluation

[Add the number and dates of the practice sessions, and who attended, such as the guide or classmates.] We evaluated each rehearsal with the criteria in Table 4.3, and after each round we changed the slides or the explanation wherever a criterion was weak.

Table 4.3: Criteria used to evaluate practice sessions

Criterion	What we checked
Timing	The talk finished within the allotted time, with time left for questions
Clarity	A listener could state the point of each slide in one sentence
Accuracy	Every number spoken matched the results tables
Delivery	Eye contact, steady pace, and no reading directly from the slides
Handover	Smooth transitions between team members
Questions	Answers were short, correct and pointed to the right slide

[Add a short summary of what improved between the first and the final rehearsal, and any feedback from the guide.]

5. Outcomes and Achievements

The project produced the following outcomes:

- A working crop recommendation system that takes seven soil and weather inputs and returns three ranked crops with probabilities; the selected Random Forest reached 99.55% accuracy on 440 unseen test rows.
- A comparison of ten machine learning algorithms on the same split, with five metrics, cross-validation and training time.
- An overfitting analysis showing a training–test gap of 0.45 percentage points for the selected model and chance-level accuracy with shuffled labels.
- An explainable AI module that explains every recommendation with SHAP and LIME, globally, per crop and for an individual farmer.
- A fertilizer module that combines an XGBoost classifier (97.69% accuracy) with a nutrient-gap rule.
- A research paper in IEEE conference format (eight pages, Appendix A), a 26-slide presentation deck, and Python code that regenerates every number and figure in this report.

No patent was filed. As explained in Section 1.2, the system is a software method that falls under the Section 3(k) exclusion of the Patents Act, so the patent route was not pursued.

5.1 Publications Accepted

The research paper described in Chapter 2 has been prepared in IEEE conference format. [State its status: submitted, under review or accepted, with the name of the conference and the date.] [Once the paper is published, add its full reference in IEEE style here.]

5.2 Awards and Recognitions

[List any awards, prizes or certificates received, for example at the project showcase or the conference. If there are none, write “None at the time of submission”.]

6. Conclusion and Future Scope

We set out to build a crop recommendation system that does more than name a crop. Our system recommends crops from seven soil and weather values, explains each recommendation, and gives fertilizer advice for the chosen crop.

Of the ten algorithms we compared on the same 80/20 split, Random Forest and Naive Bayes were the most accurate, both at 99.55% on 440 test rows, and the five best were within three test rows of each other. We chose Random Forest because it also gives a ranked list of crops and can be explained exactly with TreeSHAP. AdaBoost failed with its default one-split stumps (17.50%) but reached 99.32% with depth-3 trees, which shows how much a single setting can matter. The selected model does not overfit: its training and test accuracies differ by only 0.45 points, and with shuffled labels its accuracy falls to chance level. The high accuracy reflects how clearly the crops are separated in this dataset.

SHAP showed that humidity and rainfall have the largest overall influence on the recommendation, with N, P and K close behind, and that the deciding factor changes from crop to crop. SHAP and LIME agreed on the main reason in 66% of the test cases, so the system shows only the top two or three reasons. For fertilizer recommendation, XGBoost reached 97.69% accuracy, and the soil's N, P and K carried almost all of the predictive weight. The nutrient-gap rule turns this into advice for the crop the farmer is about to sow.

Future scope. The most important next step is to test the system on real field data. We plan to combine Soil Health Card reports with district-level rainfall and temperature records, and to check the recommendations against the crops farmers actually grow. Other directions are:

- Converting the nutrient gap into fertilizer quantities in kg/ha, using soil-test based dose recommendations, so that the advice states how much urea, DAP or MOP to apply.
- Tuning the hyperparameters of the strongest models and testing further models.
- Building a mobile application that fills in weather data automatically from the farmer's location and shows the explanation in the farmer's own language.
- Collecting feedback from farmers and extension workers on whether the explanations help them trust and act on the advice.

7. References

[1] K. G. Liakos, P. Busato, D. Moshou, S. Pearson and D. Bochtis, “Machine learning in agriculture: A review,” *Sensors*, vol. 18, no. 8, p. 2674, 2018, doi: 10.3390/s18082674.

[2] T. van Klompenburg, A. Kassahun and C. Catal, “Crop yield prediction using machine learning: A systematic literature review,” *Computers and Electronics in Agriculture*, vol. 177, p. 105709, 2020, doi: 10.1016/j.compag.2020.105709.

[3] R. Kumar, M. P. Singh, P. Kumar and J. P. Singh, “Crop selection method to maximize crop yield rate using machine learning technique,” in *Proc. Int. Conf. Smart Technologies and Management for Computing, Communication, Controls, Energy and Materials (ICSTM)*, Chennai, India, 2015, pp. 138–145.

[4] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika and J. Nisha, “Crop recommendation system for precision agriculture,” in *Proc. 8th Int. Conf. Advanced Computing (ICoAC)*, Chennai, India, 2017, pp. 32–36.

[5] Z. Doshi, S. Nadkarni, R. Agrawal and N. Shah, “AgroConsultant: Intelligent crop recommendation system using machine learning algorithms,” in *Proc. 4th Int. Conf. Computing Communication Control and Automation (ICCUBEA)*, Pune, India, 2018, pp. 1–6.

[6] Gladiator07, “Harvestify: Crop recommendation, fertilizer suggestion and plant disease prediction,” GitHub repository, commit aac427c, 2023. [Online]. Available: https://github.com/Gladiator07/Harvestify, Accessed on 11 Oct. 2026.

[7] A. Ingle, “Crop Recommendation Dataset,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset, Accessed on [date].

[8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[9] M. T. Ribeiro, S. Singh and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.

[10] M. Ryo, “Explainable artificial intelligence and interpretable machine learning for agricultural data analysis,” *Artificial Intelligence in Agriculture*, vol. 6, pp. 257–265, 2022.

[11] Government of India, *The Patents Act, 1970* (Act No. 39 of 1970).

[12] Government of India, *The Copyright Act, 1957* (Act No. 14 of 1957).

[13] Free Software Foundation, “GNU General Public License, Version 3,” 29 June 2007. [Online]. Available: https://www.gnu.org/licenses/gpl-3.0.html, Accessed on [date].

[14] “Fertilizer Prediction,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/gdabhishek/fertilizer-prediction, Accessed on [date].

[15] M. Shell, “IEEEtran: LaTeX class for IEEE journals and conferences,” CTAN. [Online]. Available: https://ctan.org/pkg/ieeetran, Accessed on [date].

[16] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[17] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[18] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[19] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[20] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[21] J. Zhu, H. Zou, S. Rosset and T. Hastie, “Multi-class AdaBoost,” *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.

[22] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[24] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[26] M. Ojala and G. C. Garriga, “Permutation tests for studying classifier performance,” *Journal of Machine Learning Research*, vol. 11, pp. 1833–1863, 2010.

8. Appendices

A. Research Paper Manuscript

The full manuscript of the research paper, “Crop Recommendation System Using Machine Learning, Explainable AI and Fertilizer Recommendation”, as prepared for submission in IEEE conference format, is reproduced on the following pages.

Crop Recommendation System Using Machine Learning, Explainable AI and Fertilizer Recommendation

Author One
Dept. of Computer Science and Engineering
University Name
City, India
author1@example.com

Author Two
Dept. of Computer Science and Engineering
University Name
City, India
author2@example.com

Guide Name
Dept. of Computer Science and Engineering
University Name
City, India
guide@example.com

***Abstract*—Choosing a crop that suits the soil and the local weather is one of the first decisions a farmer makes each season, and a poor choice costs a full season of work. We built a crop recommendation system that takes seven inputs (soil nitrogen, phosphorus, potassium and pH, temperature, humidity and rainfall), recommends the three most suitable crops, explains the recommendation, and suggests how to correct the soil with fertilizer. We trained and tested ten machine learning algorithms on a public dataset of 2,200 samples covering 22 crops, using an 80/20 stratified split. Random Forest and Naive Bayes gave the highest test accuracy, 99.55% (438 of 440 test samples), followed by AdaBoost with depth-3 trees (99.32%) and XGBoost (99.09%). AdaBoost with its default settings reached only 17.50%, and we explain why. To show the farmer why a crop was chosen, we used SHAP and LIME on the Random Forest. Humidity (22%) and rainfall (18%) had the largest overall influence, and the two methods named the same top feature for 66% of 100 test cases. For fertilizer recommendation we compared eight algorithms on a 99-sample fertilizer dataset; XGBoost was best with 97.69% accuracy under repeated cross-validation, and the soil N, P and K values carried almost all of the predictive weight. A simple nutrient-gap rule then compares the farmer’s soil with the needs of the recommended crop.**

***Index Terms*—Crop recommendation, machine learning, Random Forest, explainable AI, SHAP, LIME, fertilizer recommendation, precision agriculture**

I. Introduction

Most farmers in India decide what to sow from experience and from what their neighbours grow. That works when conditions stay the same, but soil nutrients change with years of cropping and fertilizer use, and rainfall varies from season to season. A crop that does not suit the field gives a low yield, and the loss is felt for the whole season.

Soil testing has become more common, and a farmer can now know the nitrogen (N), phosphorus (P) and potassium (K) content and the pH of the field. Weather data are also easy to obtain. The difficulty is turning these numbers into a decision. Machine learning is a natural fit here: given enough examples of the conditions under which each crop grows well, a model can learn which crop matches a new set of soil and weather values [1].

A recommendation alone is not enough, though. A farmer is unlikely to trust a crop name that comes with no reason, and the next question after “what should I grow?” is usually “what fertilizer should I apply?”. We therefore designed the system around three parts: a crop recommender, an explanation of each recommendation, and a fertilizer module.

Our contributions are:

- A comparison of ten algorithms (Logistic Regression, Naive Bayes, K-Nearest Neighbours, SVM, Decision Tree, Random Forest, XGBoost, LightGBM and two AdaBoost variants) on the same train/test split, reporting accuracy, precision, recall, F1-score, cross-validation accuracy and training time.
- An explainable AI module that uses SHAP and LIME to show which soil and weather values led to a recommendation, both for the whole dataset and for a single farmer.
- A fertilizer module with two parts: a comparison of eight algorithms that predict the fertilizer from soil and crop data, and a nutrient-gap rule that compares the farmer’s N, P and K with the requirement of the recommended crop.
- All results come from public datasets, which we list with their links so that others can repeat the work.

The rest of the paper is organised as follows. Section II covers the algorithms and related work. Section III describes the datasets and the proposed system. Section IV presents the results, and Section V concludes.

II. Background

A. Preliminary Concepts

1) Decision Tree and Random Forest: A decision tree splits the data on one feature at a time, choosing at each node the threshold that best separates the classes, until each leaf holds mostly one class [10]. A single tree is easy to read but changes a lot when the training data change slightly. Random Forest [11] trains many trees, each on a bootstrap sample of the data and with a random subset of features at each split, and takes the majority vote:

$$\hat{y} = \operatorname{mode}\{h_t(\mathbf{x})\}_{t=1}^{T}, \tag{1}$$

where h_t is the t-th tree. The fraction of trees that vote for each crop also gives a probability, which we use to rank crops.

2) K-Nearest Neighbours (KNN): KNN [12] stores the training samples and labels a new sample by a vote among the k samples closest to it. Closeness is measured with the Euclidean distance

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{M} (x_j - x'_j)^2}, \tag{2}$$

so features must be put on a common scale first; otherwise rainfall, measured in hundreds of millimetres, would dominate pH.

3) Naive Bayes: Gaussian Naive Bayes assumes that, within each crop, every feature follows a normal distribution and the features are independent. It picks the crop c that maximises

$$P(c) \prod_{j=1}^{M} \mathcal{N}\left(x_j;\, \mu_{cj}, \sigma_{cj}^2\right), \tag{3}$$

where μ_{cj} and σ_{cj} are the mean and standard deviation of feature j for crop c.

4) Support Vector Machine (SVM): An SVM finds the boundary with the widest margin between classes [13]. With a radial basis function (RBF) kernel the boundary can be curved, and multi-class problems are handled by combining several two-class SVMs.

5) Boosting: Boosting trains models one after another, each one focusing on the samples the previous ones got wrong. AdaBoost [14] raises the weight of misclassified samples after every round; its multi-class version is SAMME [15]. XGBoost [16] and LightGBM [17] are gradient boosting libraries that fit each new tree to the errors of the current ensemble and are widely used for tabular data.

6) SHAP: SHapley Additive exPlanations [18] split one prediction into a contribution from each feature:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{M} \phi_j, \tag{4}$$

where ϕ_0 is the average output of the model and ϕ_j is the contribution of feature j, computed as a Shapley value from game theory. For tree models, TreeSHAP computes these values exactly and quickly [19].

7) LIME: LIME [20] explains one prediction by creating many slightly changed copies of the input, asking the model to predict them, and fitting a simple linear model g to those predictions:

$$\xi(\mathbf{x}) = \arg\min_{g} \; \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g), \tag{5}$$

where $\pi_{\mathbf{x}}$ gives more weight to copies close to $\mathbf{x}$ and Ω keeps g simple. The weights of g are the explanation.

B. Related Work

Liakos et al. [1] reviewed machine learning in agriculture and grouped the work into crop, livestock, water and soil management; crop recommendation sits in the first group. Van Klompenburg et al. [2] reviewed crop yield prediction studies; temperature, rainfall and soil type were among the features used most often.

For recommending a crop, Kumar et al. [3] proposed a crop selection method that chooses a sequence of crops to improve the yield rate over a season. Pudumalar et al. [4] combined several classifiers in an ensemble that recommends a crop from soil properties. Doshi et al. [5] built AgroConsultant, which compares several classifiers for crop recommendation using soil and climate inputs. The public Harvestify project [7] offers crop recommendation, fertilizer suggestion and plant disease detection on one website and is the source of the crop dataset we use.

Explainable AI has started to appear in agricultural data analysis [6]. The crop recommendation studies above compare the accuracy of their models but do not explain individual recommendations to the farmer, and fertilizer advice is treated as a separate problem. Our system brings the three steps together and compares a wider set of algorithms on the same split.

III. Proposed System

A. System Overview

Fig. 1 shows the system. The farmer supplies soil test values and weather conditions. The crop model returns the three most likely crops with a probability for each. The explanation module shows which inputs pushed the model towards the first crop. The fertilizer module then checks the soil against that crop's needs.

B. Datasets

We used three public datasets (Table I).

The crop dataset was released on Kaggle by the creator of Harvestify [7], [8]. It contains 22 crops, among them rice, maize, chickpea, cotton, jute, coffee and several fruits, with exactly 100 samples per crop. The fertilizer dataset [9] maps soil and crop information to one of seven fertilizers: Urea (22 rows), DAP (18), 28-28 (17), 14-35-14 (14), 20-20 (14), 17-17-17 (7) and 10-26-26 (7). The requirement table, taken from the Harvestify repository, lists the N, P and K values that each of the 22 crops needs, on the same scale as the crop dataset.

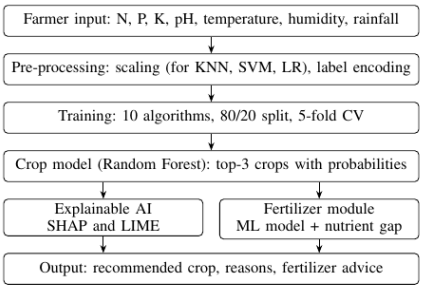


Fig. 1. Architecture of the proposed crop recommendation system.

TABLE I
DATASETS USED IN THIS WORK

Dataset	Rows	Classes	Features
Crop Recommendation [8]	2,200	22 crops (100 each)	N, P, K, temperature, humidity, pH, rainfall
Fertilizer Prediction [9]	99	7 fertilizers	temperature, humidity, moisture, soil type, crop type, N, P, K
Crop nutrient requirement [7]	22	–	required N, P, K and pH per crop

C. Data Analysis and Pre-processing

The crop dataset has no missing values and no duplicate rows. Table II gives the range of each feature. The features are on very different scales: K goes up to 205 while pH stays between 3.5 and 9.9.

TABLE II
SUMMARY OF THE CROP DATASET FEATURES

Feature	Mean	Std.	Min	Max
N	50.55	36.92	0.00	140.00
P	53.36	32.99	5.00	145.00
K	48.15	50.65	5.00	205.00
Temperature (°C)	25.62	5.06	8.83	43.68
Humidity (%)	71.48	22.26	14.26	99.98
pH	6.47	0.77	3.50	9.94
Rainfall (mm)	103.46	54.96	20.21	298.56

Fig. 2(a) shows the correlation between features. Only P and K are strongly related ($r = 0.74$); every other pair stays between -0.23 and 0.21, so each feature adds its own information. Fig. 2(b) shows how different the crops are in their nutrient needs. Apple and grapes have by far the highest P and K (around 133 and 200), cotton has the highest N (118), and crops such as mango and orange sit low on all three.

For pre-processing we encoded the crop names as integers and split the data with a stratified 80/20 split (random seed 42), giving 1,760 training and 440 test samples, 20 per crop in the test set. KNN, SVM and Logistic Regression were given standardised features (zero mean, unit variance), with the scaler fitted on the training set only. Tree models and Naive Bayes used the raw values. For the fertilizer data, soil type and crop type were one-hot encoded.

D. Algorithms and Training

Table III lists the ten algorithms and their settings. We used the library defaults apart from the settings shown, and did not tune hyperparameters. We kept AdaBoost in two forms because AdaBoost is often reported with its default settings, and those settings behave very differently on this problem (Section IV-C).

TABLE III
ALGORITHMS AND SETTINGS

Algorithm	Settings
Logistic Regression	scaled, max. 5000 iterations
Naive Bayes	Gaussian
KNN	scaled, $k = 5$
SVM	scaled, RBF kernel
Decision Tree	no depth limit
Random Forest	100 trees
XGBoost	300 trees, depth 6, learning rate 0.1
LightGBM	300 trees, 31 leaves, learning rate 0.05
AdaBoost (default)	50 decision stumps (depth 1)
AdaBoost (depth-3 trees)	200 trees of depth 3, learning rate 0.5

E. Testing and Evaluation Metrics

Each algorithm was trained on the 1,760 training samples and tested once on the 440 held-out samples. We report accuracy and the macro averages of precision, recall and F1-score over the 22 crops:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \tag{6}$$

$$F1_c = \frac{2\,\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \tag{7}$$

where TP_c, FP_c and FN_c are the true positives, false positives and false negatives for crop c. Because a single split can be lucky or unlucky, we also ran 5-fold stratified cross-validation on the training set and report its mean and standard deviation. Training time was measured on a 4-core CPU.

F. Explainable AI Module

We explained the Random Forest with TreeSHAP. The global importance of feature j is the mean absolute SHAP value over the n test samples and the $K = 22$ crop outputs:

$$I_j = \frac{1}{nK} \sum_{i=1}^{n} \sum_{k=1}^{K} \left| \phi_j^{(k)}(\mathbf{x}_i) \right|. \tag{8}$$

For each crop we also computed the importance using only that crop's own test samples and its own output, which shows what the model looks at when it recommends that crop. For a single farmer we show the SHAP values of the recommended crop next to a LIME explanation built from 2,000 perturbed samples. To see how often the two methods agree, we compared them on 100 randomly chosen test samples.

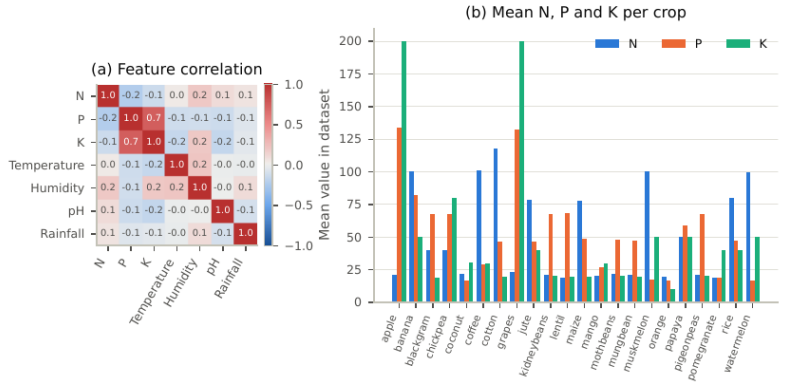


Fig. 2. Data analysis of the crop dataset. (a) Pearson correlation between the seven features. (b) Mean N, P and K for each crop.

G. Fertilizer Recommendation Module

1) Machine learning model: We trained eight algorithms to predict the fertilizer name from the eight inputs of the fertilizer dataset. With only 99 rows, a single 80/20 split would leave about 20 test rows and just one or two for the smallest classes, so we used stratified 5-fold cross-validation repeated 10 times and report the mean and standard deviation of accuracy over the 50 test folds. To see which inputs the model uses, we computed permutation importance for the Random Forest on held-out folds: the drop in accuracy when one feature's values are shuffled.

2) Nutrient-gap rule: The second part follows the approach of Harvestify [7]. For a recommended crop c with required nutrients $\mathbf{r}_c = (N, P, K)$ and the farmer's soil values $\mathbf{s}$, we compute the gap

$$\mathbf{g} = \mathbf{r}_c - \mathbf{s}, \qquad j^* = \arg\max_j |g_j|. \tag{9}$$

The nutrient j^* with the largest gap decides the advice. If $g_{j^*} > 0$ the soil is short of that nutrient and we suggest a fertilizer that supplies it: urea for N, DAP or single super phosphate (SSP) for P, and muriate of potash (MOP) for K. If $g_{j^*} < 0$ the soil already has more than the crop needs and we advise skipping that nutrient for the season.

IV. Results and Discussion

A. Experimental Setup

All experiments were written in Python 3.13 with scikit-learn 1.9.1 [21], XGBoost 3.4.1, LightGBM 4.7.0, SHAP 0.52.0 and LIME 0.2.0.1, and ran on a CPU. The random seed was 42 throughout.

B. Comparison of Algorithms

Table IV and Fig. 3 give the results on the 440 test samples. Random Forest and Naive Bayes share the top place with 99.55% accuracy; each misclassified only 2 of the 440 test samples. AdaBoost with depth-3 trees (3 errors), XGBoost (4) and LightGBM (5) follow closely. SVM, Decision Tree, KNN and Logistic Regression are a little behind, between 97.27% and 98.41%.

The differences among the top five algorithms are small. Between 99.55% and 98.86% lie only three test samples, and the cross-validation results (98.75% to 99.49%) place the same five algorithms at the top. On this dataset several algorithms work about equally well, and the choice between them should rest on other qualities. We chose Random Forest for the system. It ties for the best accuracy, trains in a quarter of a second, gives a probability for every crop so that we can show a top-3 list, and TreeSHAP explains it exactly. Naive Bayes is an equally accurate and even faster alternative when explanations are not needed.

The one clear failure is AdaBoost with default settings, at 17.50% on the test set. Its default weak learner is a stump, a tree with a single split. One split can only divide the 22 crops into two groups, so each round of boosting adds very little, and the ensemble never separates most crops. When the stump is replaced by a depth-3 tree, the same algorithm reaches 99.32%. A low AdaBoost score on this kind of problem points to the choice of weak learner, not to boosting itself.

C. Testing the Selected Model

Fig. 4 shows the confusion matrix of the Random Forest on the test set. Twenty of the 22 crops were classified perfectly. The two errors were one rice sample predicted as jute and one blackgram sample predicted as maize, which gives these four crops an F1-score between 0.974 and 0.976. Both mistakes are understandable. Rice and jute have almost the same average N, P and K (about 80, 47 and 40) and both grow in humid conditions (humidity around 80%), differing mainly in rainfall (236 mm against 175 mm). Blackgram and maize have the same average humidity (65%) and similar K.

TABLE IV
CROP RECOMMENDATION: RESULTS ON THE 440 TEST SAMPLES AND 5-FOLD CROSS-VALIDATION (CV) ON THE TRAINING SET

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Errors (of 440)	CV accuracy (%)	Training time (s)
Random Forest	**99.55**	99.57	99.55	99.55	2	99.38 ± 0.61	0.24
Naive Bayes	**99.55**	99.59	99.55	99.54	2	99.49 ± 0.42	<0.01
AdaBoost (depth-3 trees)	99.32	99.37	99.32	99.30	3	98.75 ± 0.29	0.92
XGBoost	99.09	99.13	99.09	99.08	4	99.26 ± 0.14	1.02
LightGBM	98.86	98.94	98.86	98.86	5	99.03 ± 0.39	1.44
SVM (RBF)	98.41	98.56	98.41	98.40	7	97.90 ± 1.03	0.16
Decision Tree	97.95	98.06	97.95	97.94	9	98.52 ± 0.68	0.01
KNN ($k = 5$)	97.95	98.04	97.95	97.93	9	96.53 ± 1.21	<0.01
Logistic Regression	97.27	97.40	97.27	97.25	12	96.82 ± 0.66	0.04
AdaBoost (default)	17.50	11.21	17.50	8.56	363	31.93 ± 5.85	0.14

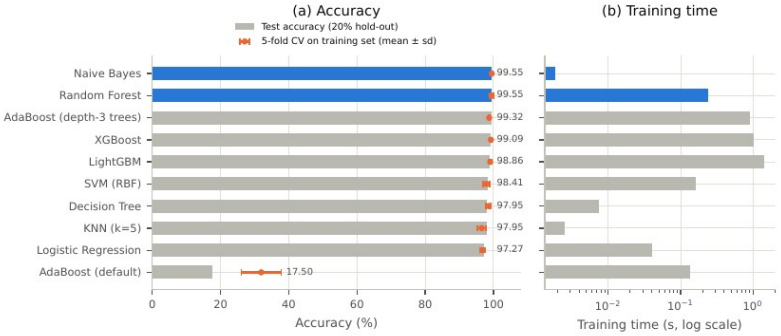


Fig. 3. Comparison of the ten algorithms. (a) Test accuracy (bars) and 5-fold cross-validation accuracy on the training set (dots, mean ± standard deviation); the two best models are highlighted. (b) Training time on a log scale.

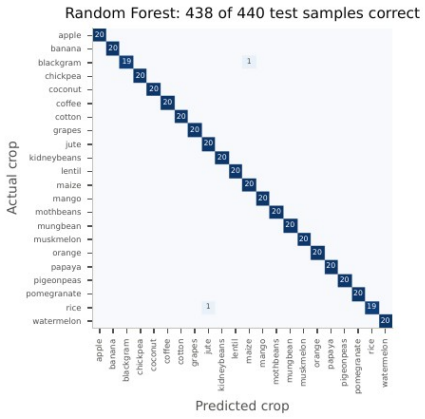


Fig. 4. Confusion matrix of the Random Forest on the 440 test samples.

Table V shows how the system responds to six farmers, taken from the test set with different crops. For all six the first recommendation matched the crop actually grown. The probabilities show how confident the model is. For F3 and F6 it is almost certain (0.99), while for F2 the first choice, moth beans, has 0.72 and pigeon peas and maize follow with 0.10 and 0.08. In such a case the top-3 list is more useful to the farmer than a single answer, since the soil and weather suit more than one crop.

D. Explainable AI: Why a Crop Is Recommended

Fig. 5(a) shows the global SHAP importance. Humidity has the largest influence (22% of the total), followed by rainfall (18%), K and N (17% each) and P (16%). Temperature (7%) and pH (3%) matter much less. The model therefore relies on a mix of weather and soil, not on one dominant feature.

The per-crop view in Fig. 5(b) is more useful to a farmer, because the reason differs from crop to crop. Rainfall is the main factor for 8 crops, including rice, jute, lentil and watermelon, and humidity for 7, including chickpea, kidney beans and mango. For apple, banana and grapes the main factor is P; for cotton and maize it is N; for orange and pomegranate it is K. These match the nutrient profiles in

TABLE V
RECOMMENDATIONS FOR SIX FARMERS FROM THE TEST SET (RANDOM FOREST; PROBABILITY IN BRACKETS)

Farmer	N	P	K	Temp.	Hum.	pH	Rain	Actual crop	1st choice	2nd choice	3rd choice
F1	18	17	31	31.7	45.2	5.7	93.8	mango	mango (0.98)	orange (0.01)	pigeon peas (0.01)
F2	22	44	24	24.3	56.3	6.0	59.0	moth beans	moth beans (0.72)	pigeon peas (0.10)	maize (0.08)
F3	3	67	24	17.0	19.9	5.5	103.3	kidney beans	kidney beans (0.99)	pigeon peas (0.01)	blackgram (0.00)
F4	34	75	24	23.5	51.3	4.8	192.3	pigeon peas	pigeon peas (0.98)	moth beans (0.02)	blackgram (0.00)
F5	76	54	45	24.3	77.6	6.2	185.0	jute	jute (0.91)	rice (0.08)	papaya (0.01)
F6	117	76	47	25.6	77.4	6.1	93.1	banana	banana (0.99)	cotton (0.01)	blackgram (0.00)

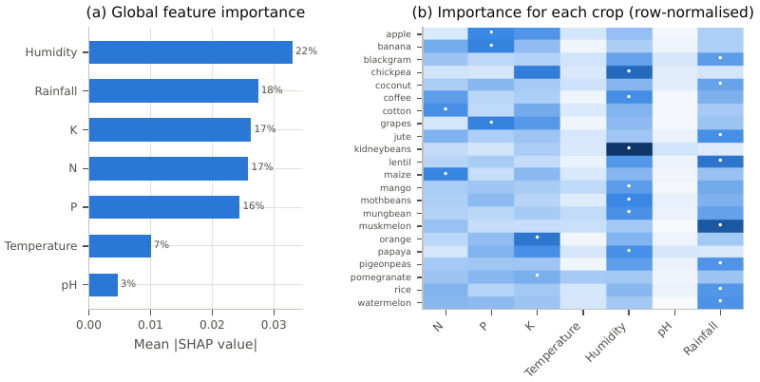


Fig. 5. SHAP explanation of the Random Forest. (a) Global importance (mean absolute SHAP value; share of the total in per cent). (b) Importance of each feature when the model recommends each crop, normalised per row; the dot marks the most important feature for that crop.

Fig. 2(b): cotton stands out by its high N, and apple and grapes by their high P and K.

Fig. 6 explains the recommendation for farmer F1, who was advised to grow mango with probability 0.98. SHAP gives the largest contributions to humidity (45.2%, contribution 0.23) and rainfall (93.8 mm, 0.20), followed by K, P and N. In plain words, the field is fairly dry and warm (31.7°C) and the soil is low in all three nutrients, which is the typical setting for mango in the data. LIME also puts humidity first and reports the rule "humidity $\leq$ 60.2", but ranks P second and rainfall fifth. The two methods agree on the overall picture but not on every detail. Over 100 test samples they named the same top feature in 66% of cases, and on average they shared 78% of their top three features. Showing the farmer only the first two or three reasons, where the methods mostly agree, is the safer choice.

E. Fertilizer Recommendation

Table VI and Fig. 7(a) compare the eight algorithms on the fertilizer dataset. XGBoost gave the highest accuracy, 97.69%, followed by Decision Tree (96.28%), Random Forest (95.07%) and Logistic Regression (94.35%). KNN (71.41%) and Naive Bayes (66.74%) did poorly. Both treat all eight inputs as equally relevant, while only three of them turn out to matter (below), and two classes have just seven rows each. The standard deviations are larger than for the crop data (3 to 12 points) because each test fold holds only about 20 rows.

TABLE VI
FERTILIZER RECOMMENDATION: ACCURACY AND MACRO F1-SCORE OVER 5-FOLD CROSS-VALIDATION REPEATED 10 TIMES

Algorithm	Accuracy (%)	F1-score (%)
XGBoost	**97.69 ± 3.20**	95.02
Decision Tree	96.28 ± 4.00	92.24
Random Forest	95.07 ± 4.43	89.22
Logistic Regression	94.35 ± 4.56	87.59
SVM (RBF)	88.09 ± 5.19	77.61
AdaBoost (depth-3 trees)	86.19 ± 12.09	74.14
KNN ($k = 5$)	71.41 ± 10.40	63.13
Naive Bayes	66.74 ± 9.59	60.58

The feature importance in Fig. 7(b) explains the result. Shuffling phosphorus lowers the Random Forest's accuracy by 0.44, nitrogen by 0.31 and potassium by 0.26, while shuffling crop type, temperature, moisture, soil type or humidity changes it by 0.006 or less. In this dataset the choice of fertilizer depends almost entirely on the soil N, P and K, which makes sense, since a fertilizer is chosen to supply the nutrients the soil lacks. In one 5-fold run XGBoost misclassified 2 of 99 samples (Fig. 7(c)): one 10-26-26 sample predicted as 14-35-14 and one 17-17-17 sample predicted as 20-20, both pairs of

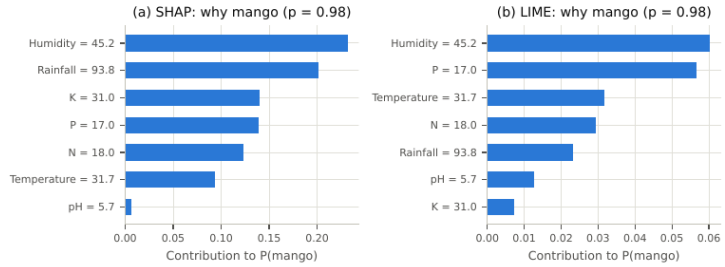


Fig. 6. Explanation of one recommendation (farmer F1, mango). (a) SHAP contribution of each input to the probability of mango. (b) LIME weights for the same input.

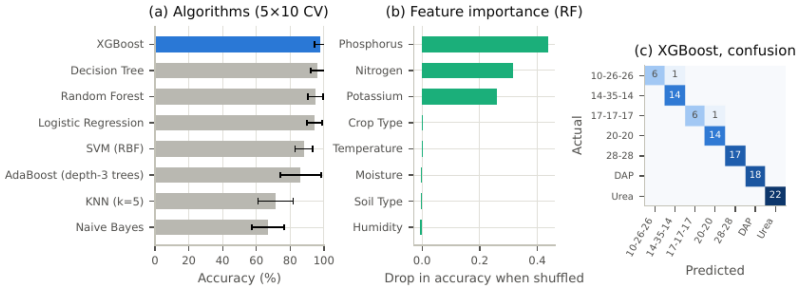


Fig. 7. Fertilizer recommendation. (a) Accuracy of the eight algorithms (mean ± standard deviation over 50 test folds). (b) Permutation importance of each input for the Random Forest, on held-out folds. (c) Confusion matrix of XGBoost over one 5-fold run.

complex fertilizers that supply all three or two of the nutrients.

Table VII applies the nutrient-gap rule to the six farmers of Table V, for the first and the second recommended crop. For the first crop the gaps are mostly small. F1's largest gap for mango is only 3 units of P, and F2's is 4 units for moth beans. This is expected, because the crop model picks the crop whose usual conditions are closest to the farmer's field. The second crop usually needs more correction: F1's soil has 21 units more K than orange needs, and F6's soil has 36 units more P than cotton needs. F3 is an exception. The soil has only 3 units of N against the 20 that kidney beans need, so the system advises adding a nitrogen fertilizer such as urea. The rule thus also tells the farmer how much the soil would have to change for a crop they prefer.

F. Discussion

Three points come out of the results. First, on this dataset the best algorithms are very close, and Random Forest, Naive Bayes, XGBoost and AdaBoost with proper weak learners all exceed 99%. Reporting only the single highest number would hide this; the cross-validation and the error counts show it clearly. Second, the explanations make the recommendations easier to check. The per-crop importance agrees with what the data show about each crop, and a farmer can see which of their own values led to the advice. SHAP and LIME do not always agree, so we treat the top two or three reasons as the explanation and do not rank every feature. Third, the fertilizer models confirm that soil N, P and K drive the fertilizer choice, and the nutrient-gap rule turns this into advice tied to the crop the farmer is about to sow.

Limitations. The crop dataset was assembled by its author, who combined typical nutrient values for each crop with weather records and notes that it should not be used for real farming decisions [7]. Our accuracies therefore show how well the algorithms learn this dataset; they are not yet a measure of performance on farmers' fields, which needs testing with real soil test reports. The fertilizer dataset has only 99 rows, and two of its classes have seven rows each. Neither dataset states the units of N, P and K, so the nutrient-gap advice names a nutrient and a direction, not a quantity in kg/ha. Finally, all algorithms used default settings; tuning could change the order among the closely matched top models.

V. Conclusion and Future Work

We built a crop recommendation system that recommends a crop, explains the recommendation and suggests a fertilizer. Of the ten algorithms we tested, Random Forest and Naive Bayes were the most accurate, both at 99.55% on 440 test

TABLE VII
NUTRIENT-GAP ADVICE FOR THE SIX FARMERS (GAP = CROP REQUIREMENT − SOIL VALUE; THE LARGEST GAP DECIDES THE ADVICE)

Farmer	Soil N-P-K	Rank	Crop	Required N-P-K	Largest gap	Advice
F1	18-17-31	1	mango	20-20-30	P low by 3	add phosphate (DAP or SSP)
		2	orange	20-10-10	K high by 21	skip potash this season
F2	22-44-24	1	moth beans	20-40-20	P high by 4	skip phosphate this season
		2	pigeon peas	20-60-20	P low by 16	add phosphate (DAP or SSP)
F3	3-67-24	1	kidney beans	20-60-20	N low by 17	add nitrogen (urea)
		2	pigeon peas	20-60-20	N low by 17	add nitrogen (urea)
F4	34-75-24	1	pigeon peas	20-60-20	P high by 15	skip phosphate this season
		2	moth beans	20-40-20	P high by 35	skip phosphate this season
F5	76-54-45	1	jute	80-40-40	P high by 14	skip phosphate this season
		2	rice	80-40-40	P high by 14	skip phosphate this season
F6	117-76-47	1	banana	100-75-50	N high by 17	skip nitrogen this season
		2	cotton	120-40-20	P high by 36	skip phosphate this season

samples, and the five best were within three test samples of each other. AdaBoost failed with its default stumps (17.50%) but reached 99.32% with depth-3 trees. SHAP showed that humidity and rainfall have the largest overall influence, with N, P and K close behind, and that the deciding factor changes from crop to crop. For fertilizer recommendation, XGBoost reached 97.69% and the soil N, P and K values carried almost all of the importance.

In future work we plan to test the system on real Soil Health Card reports combined with district rainfall records, so that the recommendations can be checked against what farmers actually grow. We also plan to convert the nutrient gap into fertilizer quantities in kg/ha using soil test based dose recommendations, to tune the hyperparameters of the strongest models, and to build a mobile application that gives the explanation in the farmer's own language.

REFERENCES

[1] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.

[2] T. van Klompenburg, A. Kassahun, and C. Catal, "Crop yield prediction using machine learning: A systematic literature review," *Computers and Electronics in Agriculture*, vol. 177, p. 105709, 2020.

[3] R. Kumar, M. P. Singh, P. Kumar, and J. P. Singh, "Crop selection method to maximize crop yield rate using machine learning technique," in *Proc. Int. Conf. Smart Technologies and Management for Computing, Communication, Controls, Energy and Materials (ICSTM)*, 2015, pp. 138–145.

[4] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika, and J. Nisha, "Crop recommendation system for precision agriculture," in *Proc. 8th Int. Conf. Advanced Computing (ICoAC)*, 2017, pp. 32–36.

[5] Z. Doshi, S. Nadkarni, R. Agrawal, and N. Shah, "AgroConsultant: Intelligent crop recommendation system using machine learning algorithms," in *Proc. 4th Int. Conf. Computing Communication Control and Automation (ICCUBEA)*, 2018, pp. 1–6.

[6] M. Ryo, "Explainable artificial intelligence and interpretable machine learning for agricultural data analysis," *Artificial Intelligence in Agriculture*, vol. 6, pp. 257–265, 2022.

[7] A. Ingle (Gladiator07), "Harvestify: Crop and fertilizer recommendation and plant disease detection," GitHub repository, 2023. [Online]. Available: https://github.com/Gladiator07/Harvestify

[8] A. Ingle, "Crop recommendation dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset

[9] "Fertilizer prediction," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/gdabhishek/fertilizer-prediction

[10] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[14] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[15] J. Zhu, H. Zou, S. Rosset, and T. Hastie, "Multi-class AdaBoost," *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[17] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[19] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[20] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[21] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

B. Event Photographs and Reports

[Insert photographs of the project showcase and any event reports or certificates here, each with a short caption giving the event, the date and the people shown.]